



SnowPro Specialty: Snowpark Certification Exam

Course and Practice Exams, to help you become a Specialist in Snowpark with the new Snowflake Certifications



About the SnowPro Snowpark Certification

SnowPro Specialty: Snowpark Certification and Exam

- SPS-B01 beta exam - between Oct 28th .. Dec 20th 2024
- announced on Oct 21st, with the Native Apps specialty certification
- results received 8-9 weeks after the window closed (~beginning of Mar 2025)
- for Data Engineers, AI/ML Engineers, Data Scientists, Data Application Developers, Data Analysts with programming experience
- SnowPro Core required
- cost (per attempt) → US \$112 for beta, US \$225 when live
- scenario-based questions, interactive questions, real-world examples
- ~2 hours, ~80 questions? ~75% passing score? (as for Advanced?)

What is NOT Included

- Snowpark Container Services (SPCS)
- Snowpark in Java or Scala
- PySpark (Python for Apache Spark)
- Procedures and Functions in SQL
- Snowflake Scripting or JavaScript

- Snowpark ML (in depth!)
- pandas on Snowflake API (in depth!)

Personal Experience

- harder than expected
- 2h marathon → could run out of time (<2 mins / question)
- many long scenarios → must scroll down one screen or more
- a lot of Python code sequences → in both top text + choices
- must know in detail some unexpected topics
- some questions are tricky and kind of gotchas
- it's indeed for developers → must have Python programming skills
- must know the Snowpark Python APIs in detail
- no proper external resources for this exam other than mine ;)

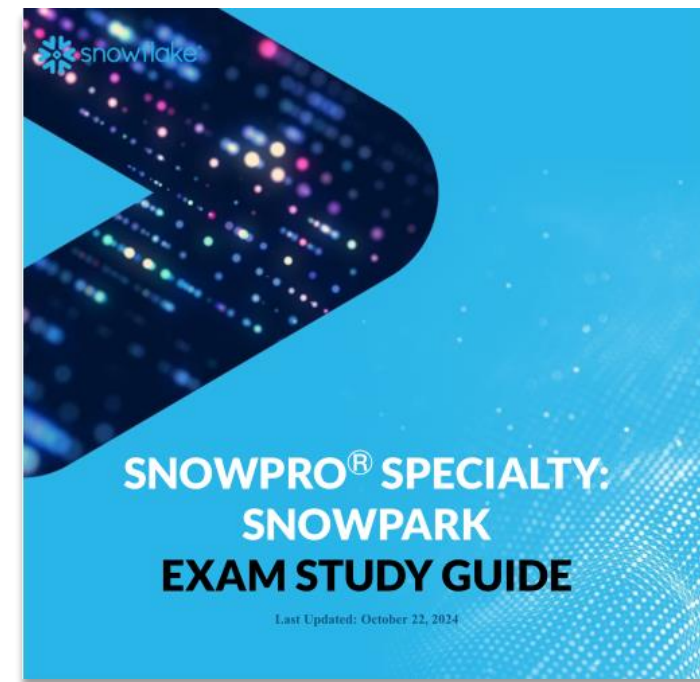


Course Structure and Content

Course Domains



- 1. Snowpark Concepts (15%)
- 2. Snowpark API for Python (30%)
- 3. Data Transformations (35%)
- 4. Performance Optimization (20%)



Course Sections

1. Snowpark DataFrame Transformations
2. Snowpark DataFrame Integrations
3. Procedures and Functions in Snowpark
4. Snowpark Configuration and Environments





About your Instructor

Cristian Scutaru



- Former Snowflake "Data Superhero"
- SnowPro Certification SME
- Passed 5x SnowPro Certification Exams
- Highly Specialized on Snowflake
- Served Dozens of Snowflake Customers
- Udemy Courses & Published Books



SnowPro Subject Matter Expert

ISSUED TO

Cristian Scutaru



Issued on: 10 JAN 2023 | Expires on: 10 JAN 2025 | Issued by: Snowflake
Verify: <https://www.credly.com/go/RGxqCsnM>





How to Benefit Most from this Course

How to Benefit Most from this Course

- Hand-on Exercises + Slideshows
- Source Code + Shared Slides
- Prerequisites + Expectations
- Captions and Playback Speed
- Q&A + Money-Back Guarantee
- Ratings and Reviews



Environment Setup

Free Snowflake Trial Account

- 30 days and/or US \$400 credits
- no credit card required
- email address you didn't use in the past 2 months
- select the Enterprise Edition
- select the AWS provider in a US region (if possible)
- account created within seconds!



- Snowpark DataFrame API Overview
- DataFrame Selections and Actions
- Filter and Transform Data
- Joins and Sets on DataFrames
- Clean and Sample Data with DataFrames
- DataFrame Functions
- DataFrame Aggregations
- Window Functions in Snowpark



Snowpark DataFrame Transformations

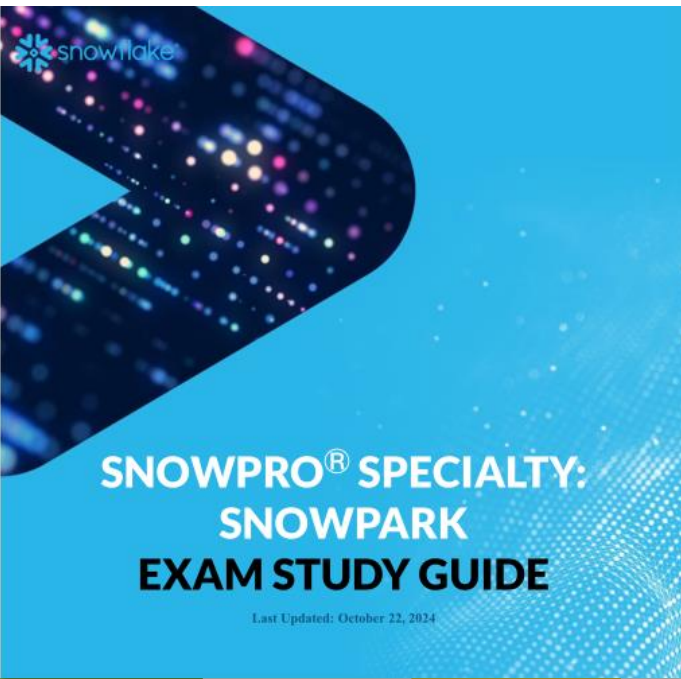


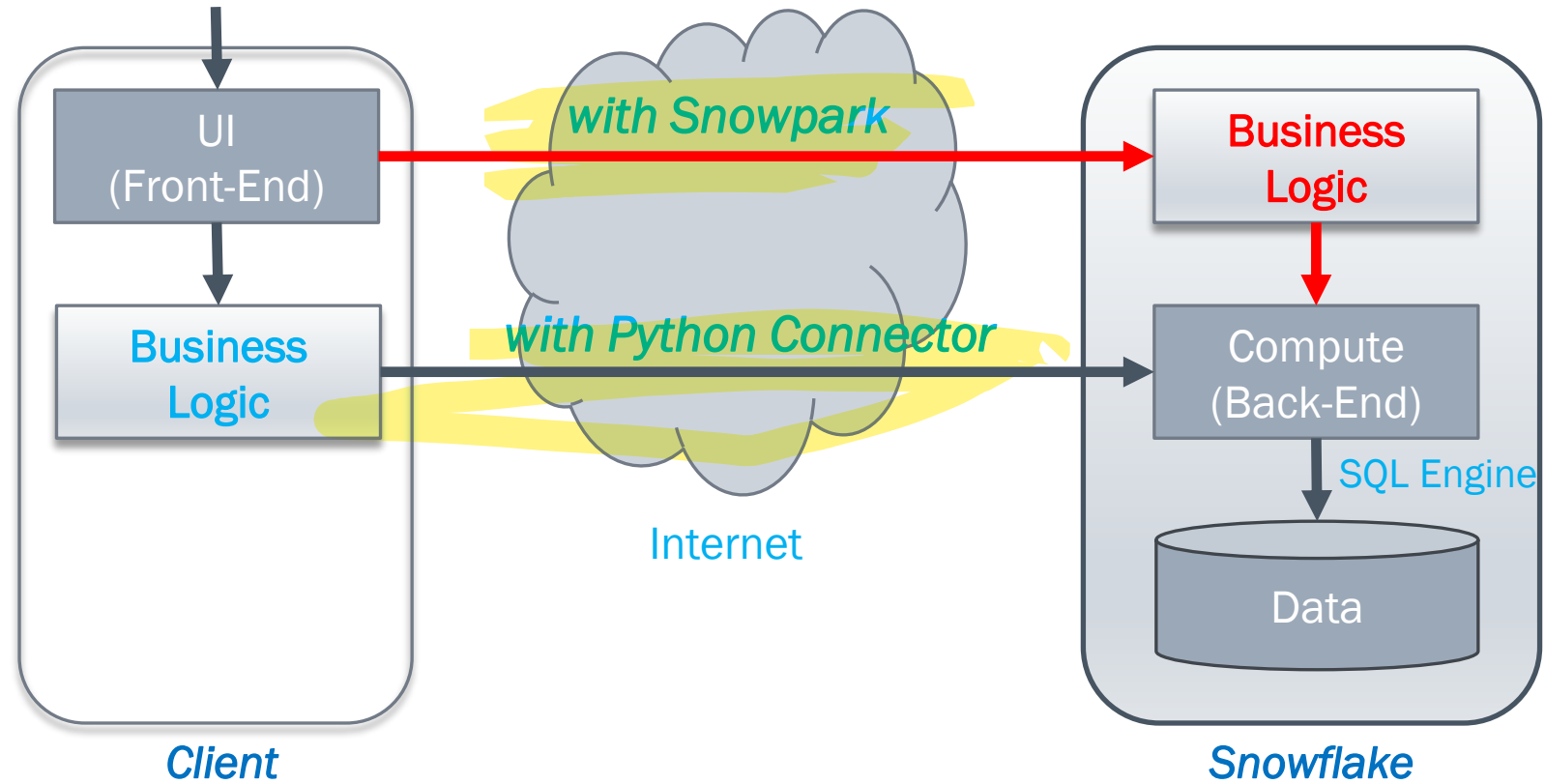
Snowpark DataFrame API Overview

What We'll Cover (from the Study Guide)

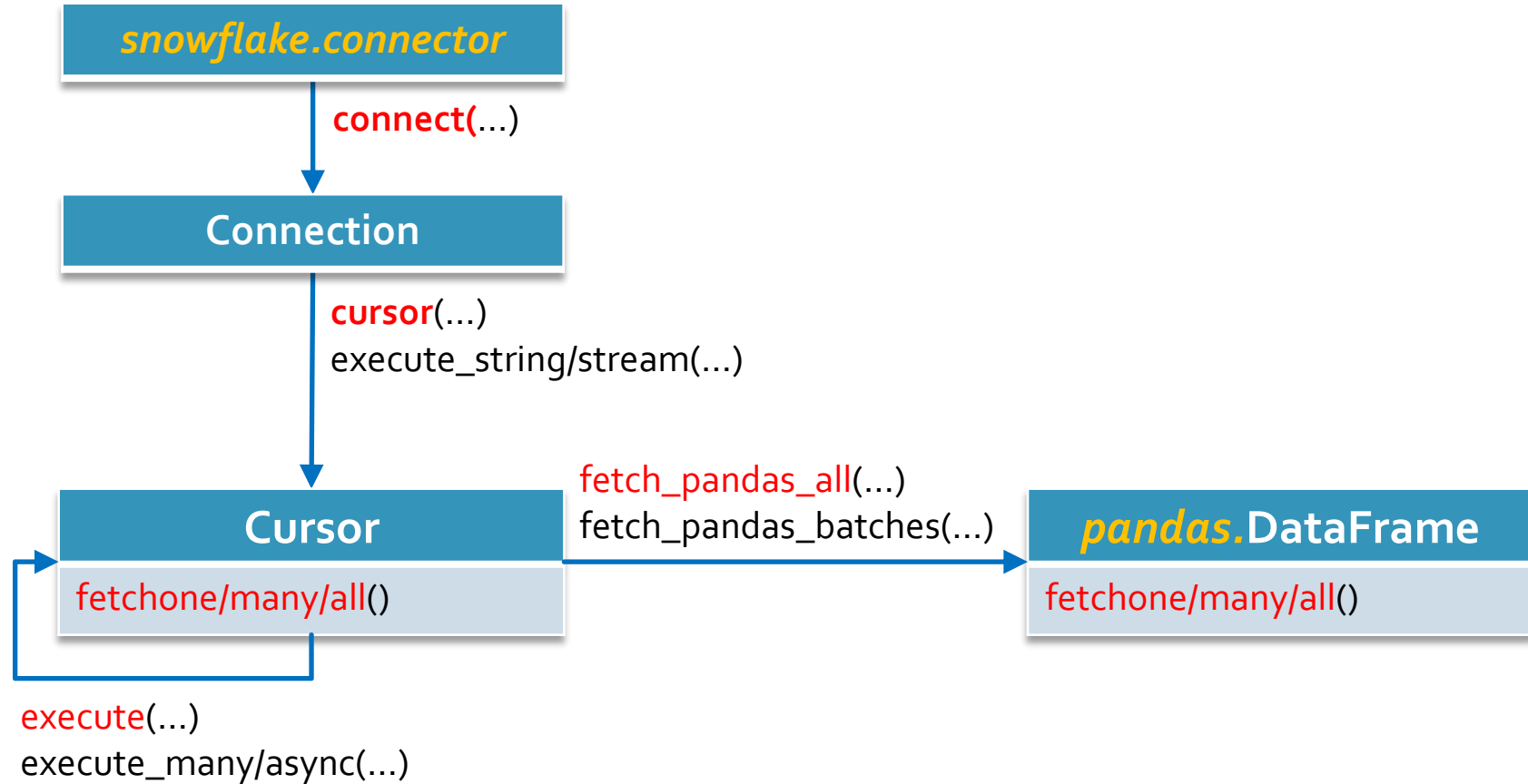
1.1 Outline Snowpark architecture

- Use of key objects
 - Snowpark DataFrames
- Types of libraries
 - DataFrames
- Client-side and server-side capabilities





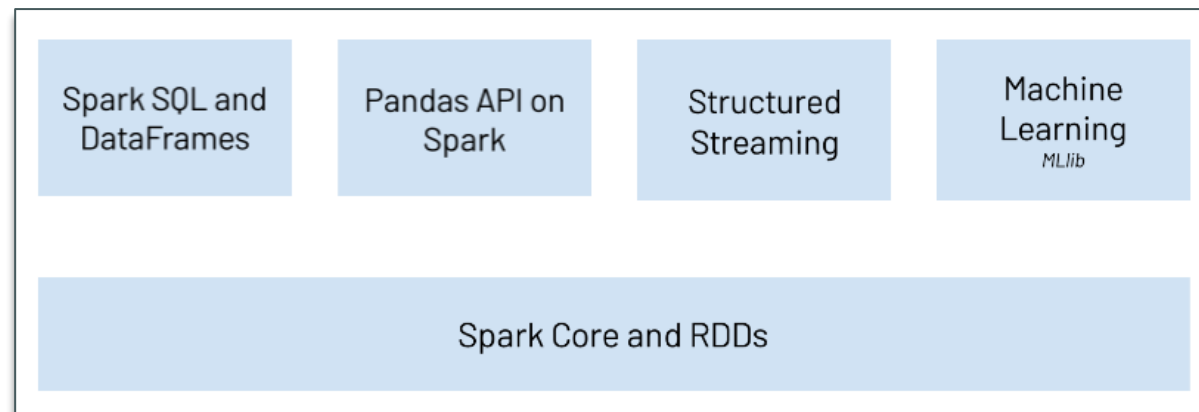
Snowpark vs Python Connector



Python Connector API

History of the Snowpark APIs

- PySpark (Python for Apache Spark) DataFrames → inspired **Snowpark DataFrame**
- pandas (Python lib) → inspired Pandas API on Spark + **pandas on Snowflake API**
- MLLib (Machine Learning in PySpark) → inspired **Snowpark ML**
- *mixedCase* (ex: groupBy) → **snake_case** (ex: group_by) names



Snowpark API: The Object Model

- Personal Blog Post from Jul 2023
- Also on the Snowflake DSH Blog
- Reposted by 40+ Snowflake employees
- Visual Class Diagrams
- Classes grouped by functionality

Snowpark API: The Object Model



Cristian Scutaru · Follow

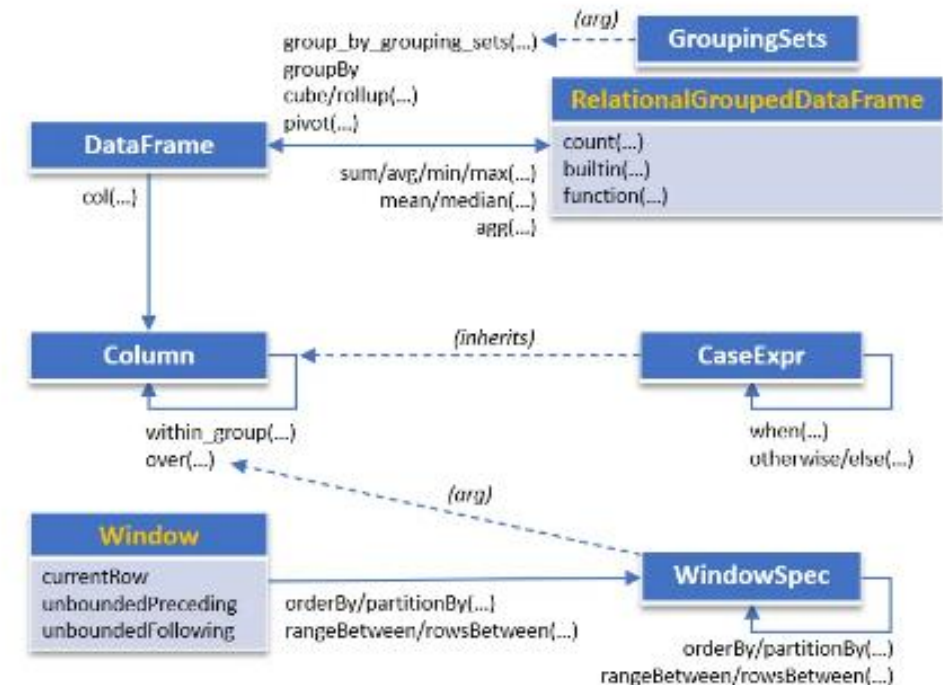
Published in Snowflake · 5 min read · Jul 11



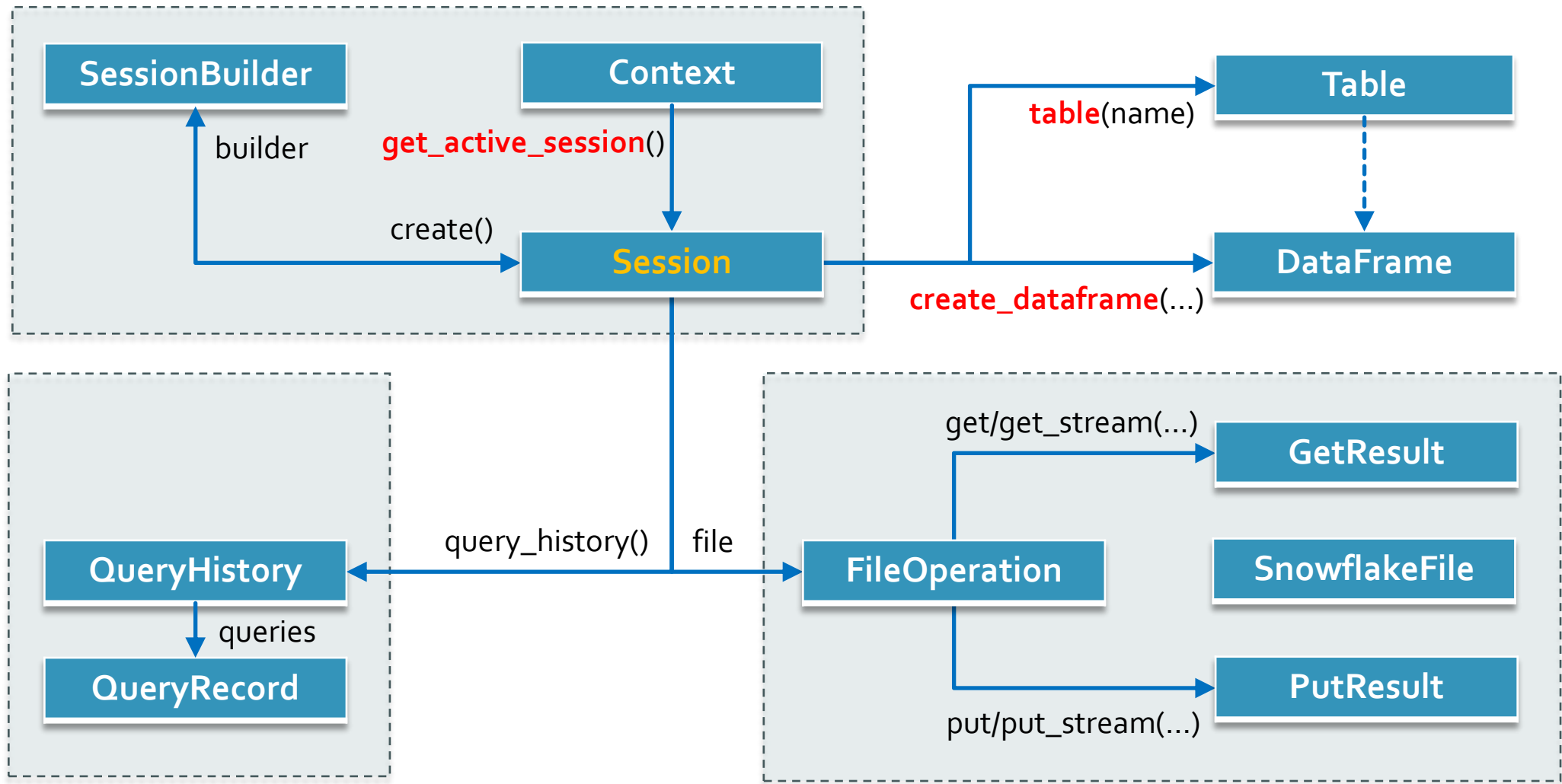
92



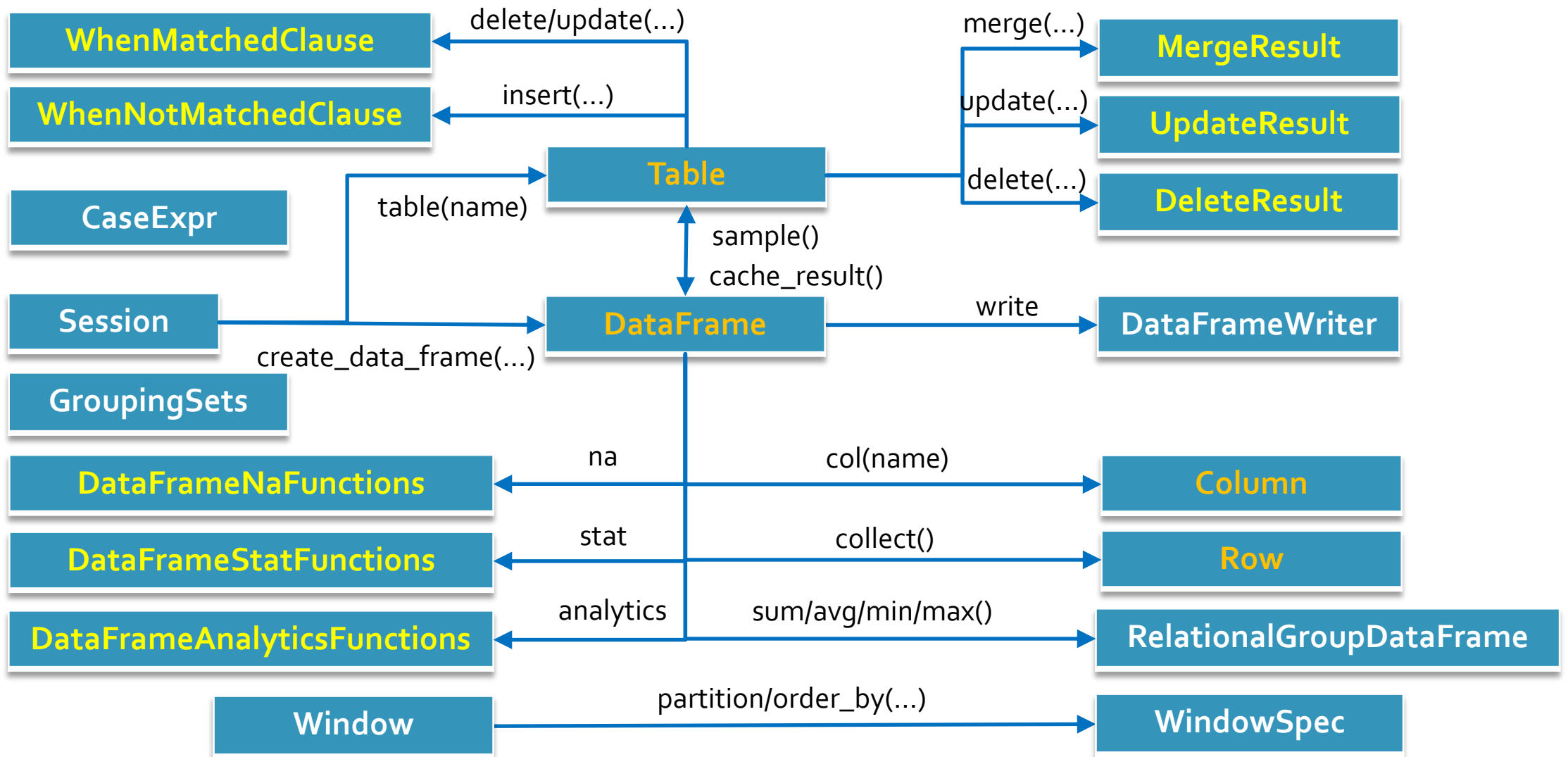
2



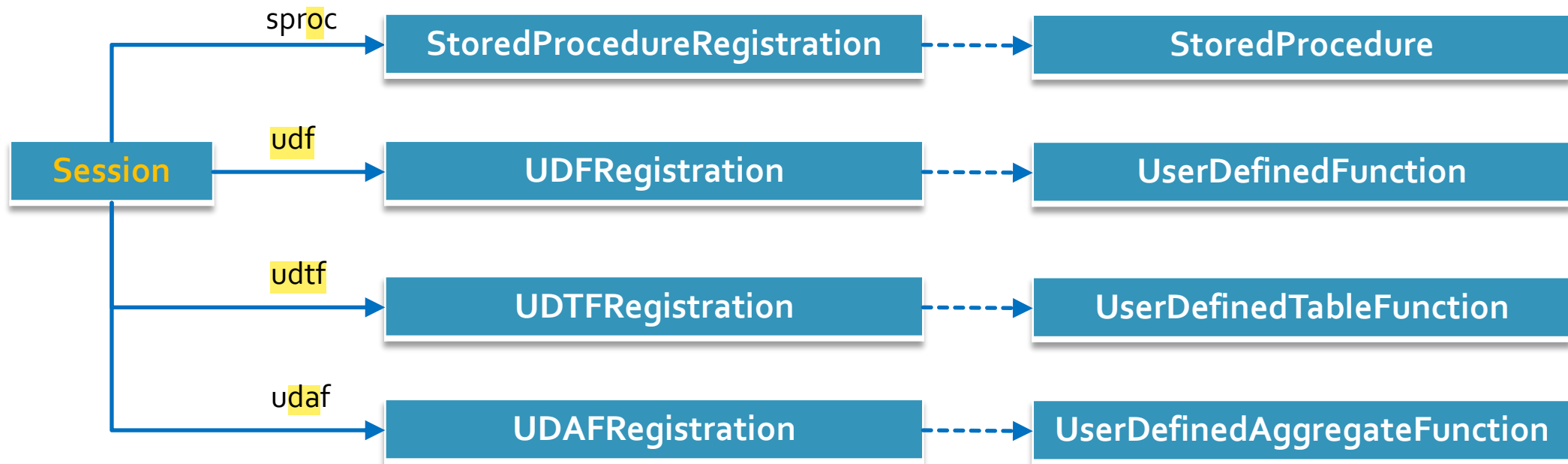
The [Snowpark API Reference for Python](#) looks great. But when you have dozens of classes with hundreds of methods and properties, some class diagrams for the whole object model will help as well.



Session Class



DataFrame Class



Procedures and Functions



DataFrame Selections and Actions

What We'll Cover (from the Study Guide)

1.1 Outline Snowpark architecture

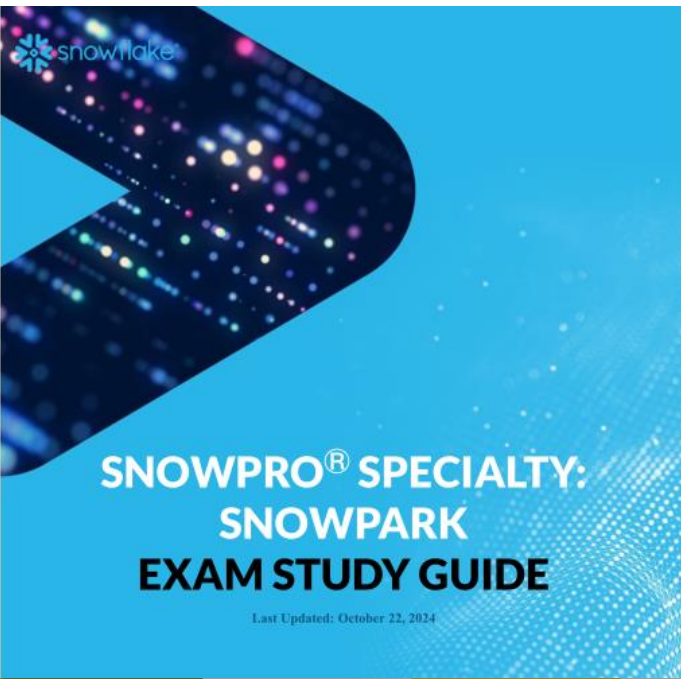
- Lazy evaluation
- Use of key objects: Snowpark DataFrames

4.2 Enhance performance in Snowpark applications

- Synchronous versus asynchronous calls: *Block* parameter

3.1 Apply operations for filtering and transforming data

- Columns
- Data type casting
- Rows and data extraction from a Rows object

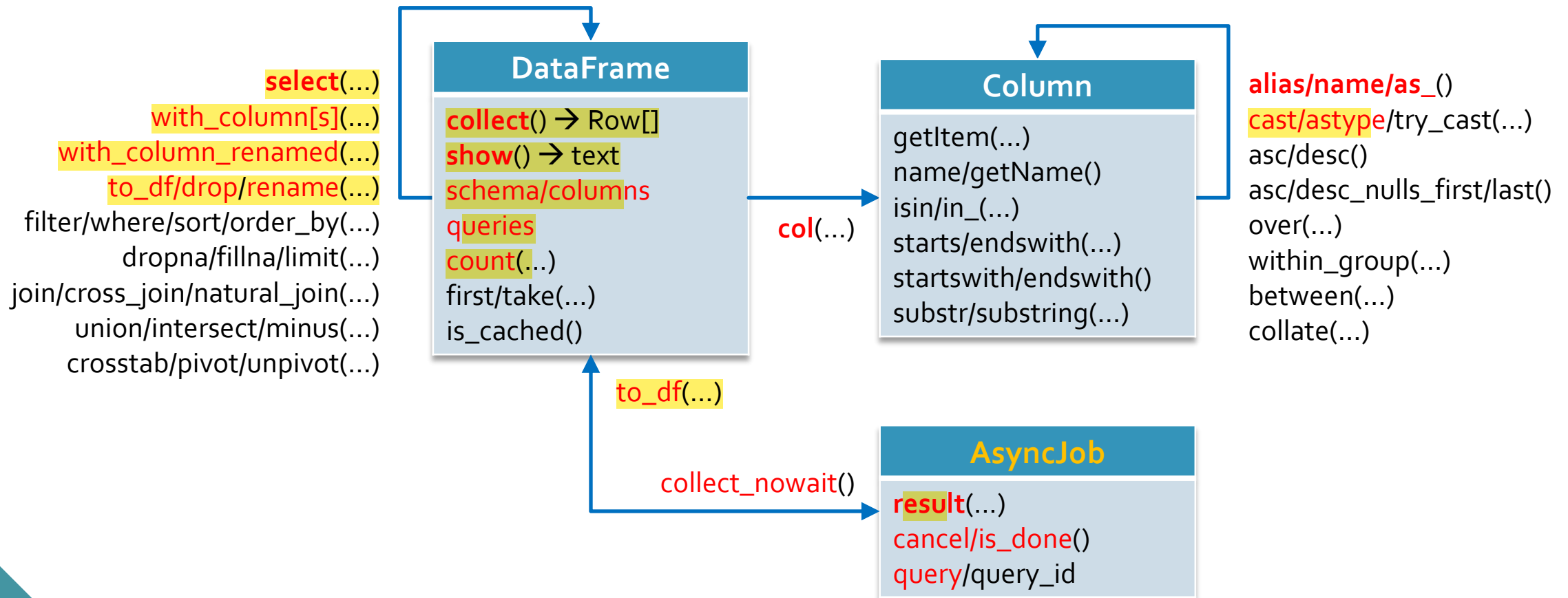


Snowpark DataFrame API Concepts

- *mixedCase* (ex: `groupBy`) → *snake_case* (ex: `group_by`) names
 - other legacy constructs: `as_` (for alias), `sort` (synonym for `order_by`)
- transform functions return a new DataFrame (no in-place changes, as in pandas!)
- *lazy evaluation*
 - particular for the Snowpark DataFrame API (~PySpark DataFrames)
 - only few "action" methods (*`show/collect/count...`*) will generate+execute an SQL query
 - `DataFrameWriter.save_as_table(...)` → also an "action" method
 - most other methods will only record internal client-side changes
 - in pandas → all loaded in memory + every call triggers internal non-SQL transformations

Column and AsyncJob Classes

- **Column**
 - "name" - column name (double-quote inside if case-sensitive)
 - col("name") - Snowpark function (from snowflake.snowpark.functions)
 - df.name, df["name"], df.col("name") - contextual to the DataFrame object
- **AsyncJob**
 - returned by DataFrame.collect_nowait(), for async processing
 - ~passing the block=False param in most action methods
 - cancel() → is_done()
 - result(...) / to_df() → to execute+return result (blocking action!)



DataFrame Selection and Action

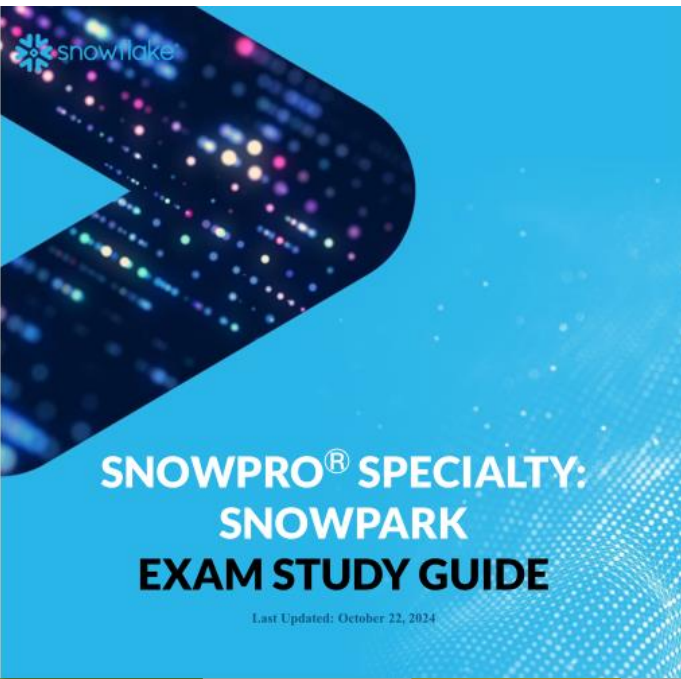


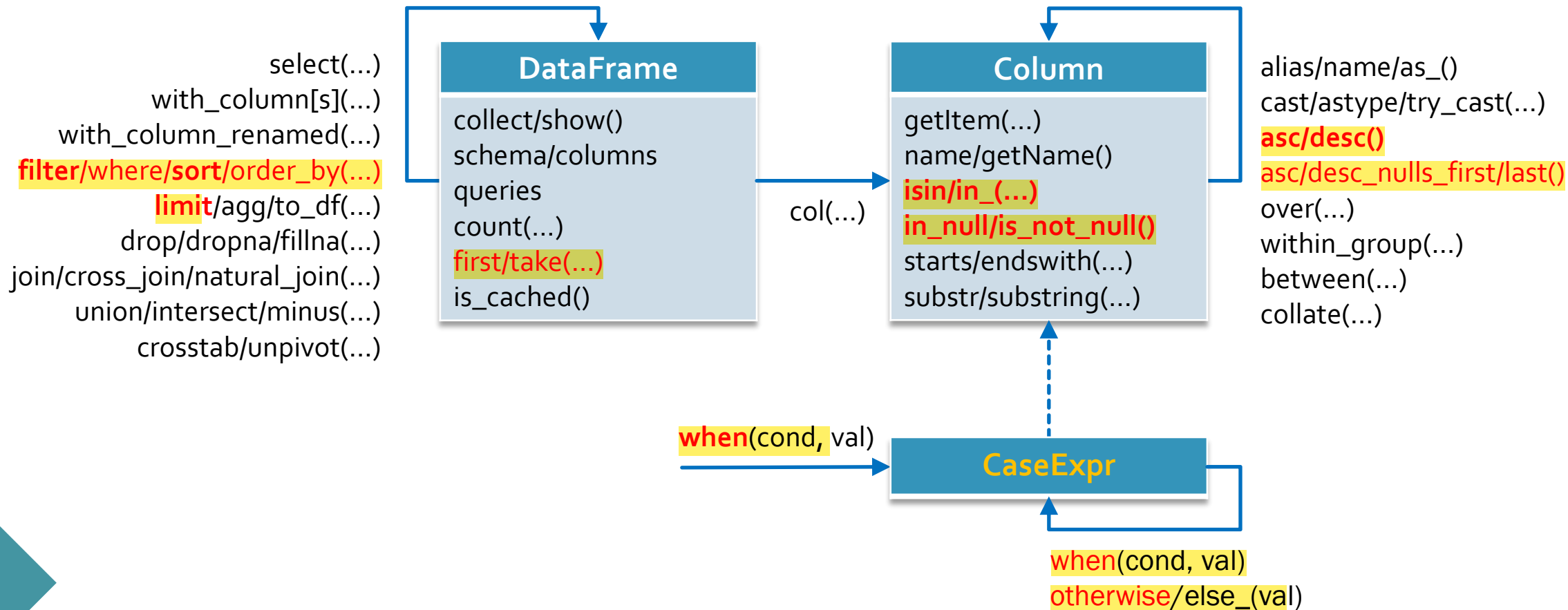
Filter and Transform Data

What We'll Cover (from the Study Guide)

3.1 Apply operations for filtering and transforming data

- Use scalar functions and operators
- Sort and limit results
- Input/output (parameters)
- Snowpark DataFrames





DataFrame Filter and Transform



Joins and Sets on DataFrames

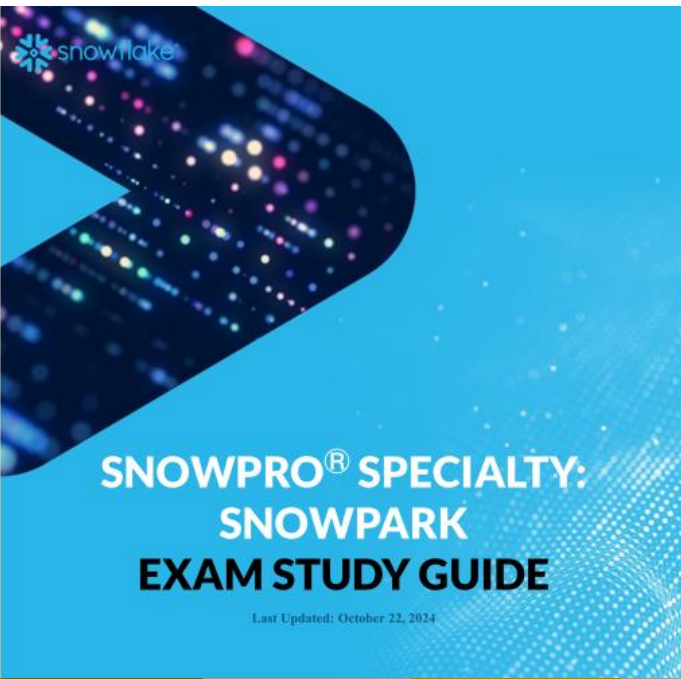
What We'll Cover (from the Study Guide)

3.2 Clean and enrich data using Snowpark for Python

- Perform joins

3.3 Perform aggregate and set-based operations on DataFrames

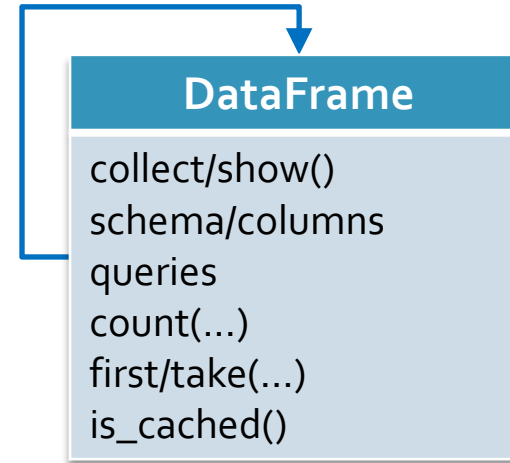
- (Union/Intersect/Except)



Joins and Sets on DataFrame Key Points

- `df = dfLeft.join(dfRight, on=..., how=...)`
 - `on` = join condition (use *match_condition* for ASOF joins)
 - `how` = join type: inner (def), left/right, full/anti, cross, asof
 - `lsuffix/rsuffix` = override suffixes for same joined column names
- `cross_join` → ~join w/ `how="cross"` (no ON)
- `natural_join` → on all columns w/ the same names (no ON)
- `join_table_function` → for LATERAL joins (w/ TABLE...)
- `copy` (clone) dataframe for self-joins
- for sets: `union[_all][_by_name]` + `intersect` + `subtract/minus/except_`

select(...)
with_column[s](...)
with_column_renamed(...)
filter/where/sort/order_by(...)
limit/agg/to_df(...)
drop/dropna/fillna(...)
join/cross_join/natural_join(...)
union/intersect/minus(...)
crosstab/unpivot(...)



Joins and Sets on DataFrame



Clean and Sample Data with DataFrames

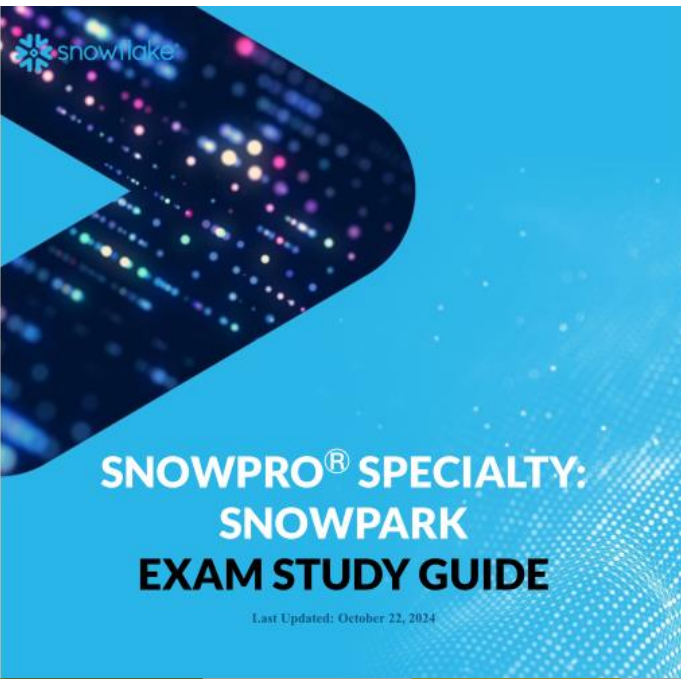
What We'll Cover (from the Study Guide)

3.2 Clean and enrich data using Snowpark for Python

- Handle missing values
- Sample data

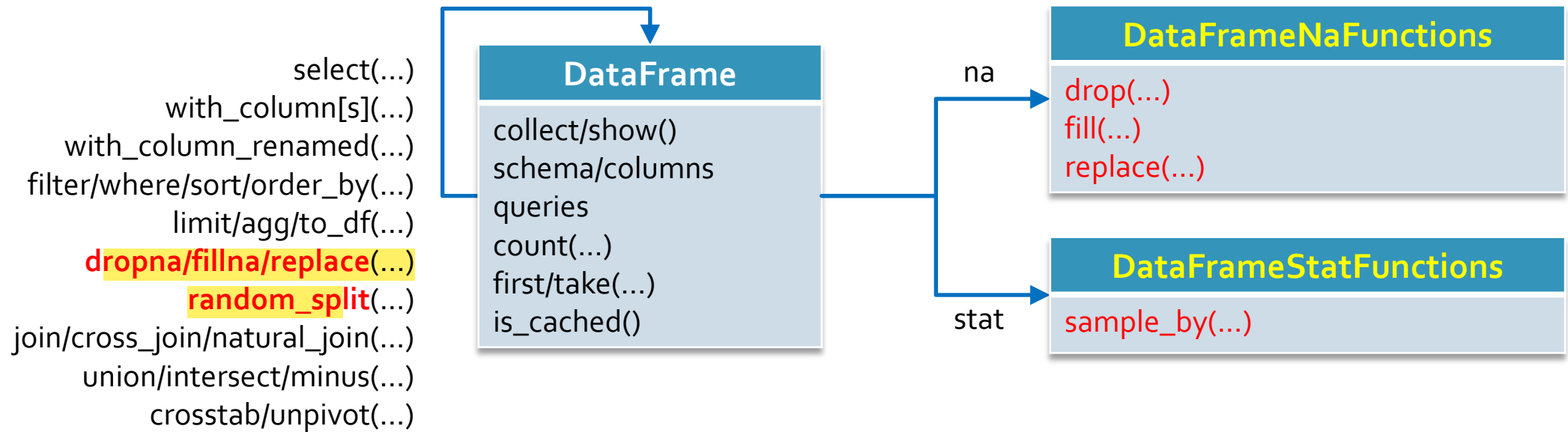
3.3 Perform aggregate and set-based operations on DataFrames

- Functions



Clean and Sample Data Key Points

- **DataFrameNaFunctions** = get w/ DataFrame.*na* → *drop/fill/replace*(...)
 - *dropna*(...) → remove rows w/ NULLs (how="all/any")
 - *fillna*(...) → replace missing values w/ something
 - *replace*() → replace/rename values
- **DataFrameStatFunctions** = get w/ DataFrame.*stat* → *sample_by*(...)
 - *sample*(...) → get a fraction of some N rows
 - *random_split*(weights, seed,...) → split into multiple DataFrames



Clean and Sample Data

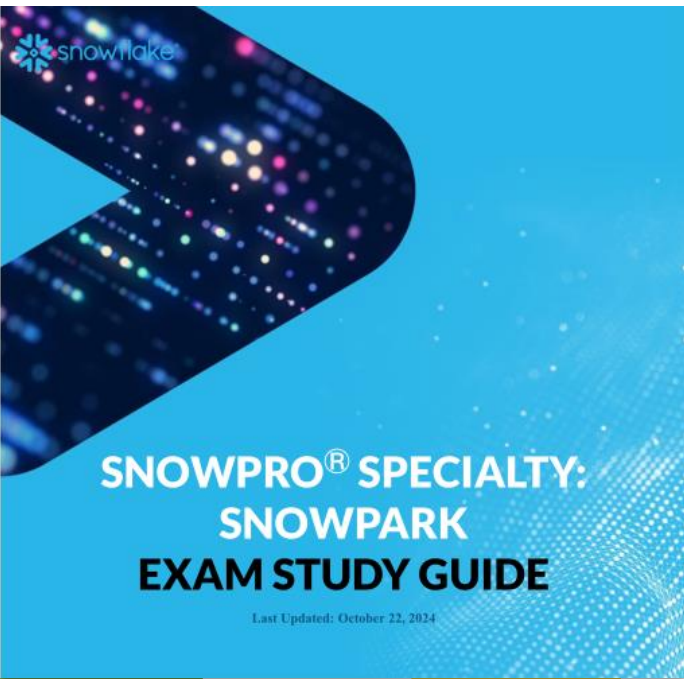


DataFrame Aggregations

What We'll Cover (from the Study Guide)

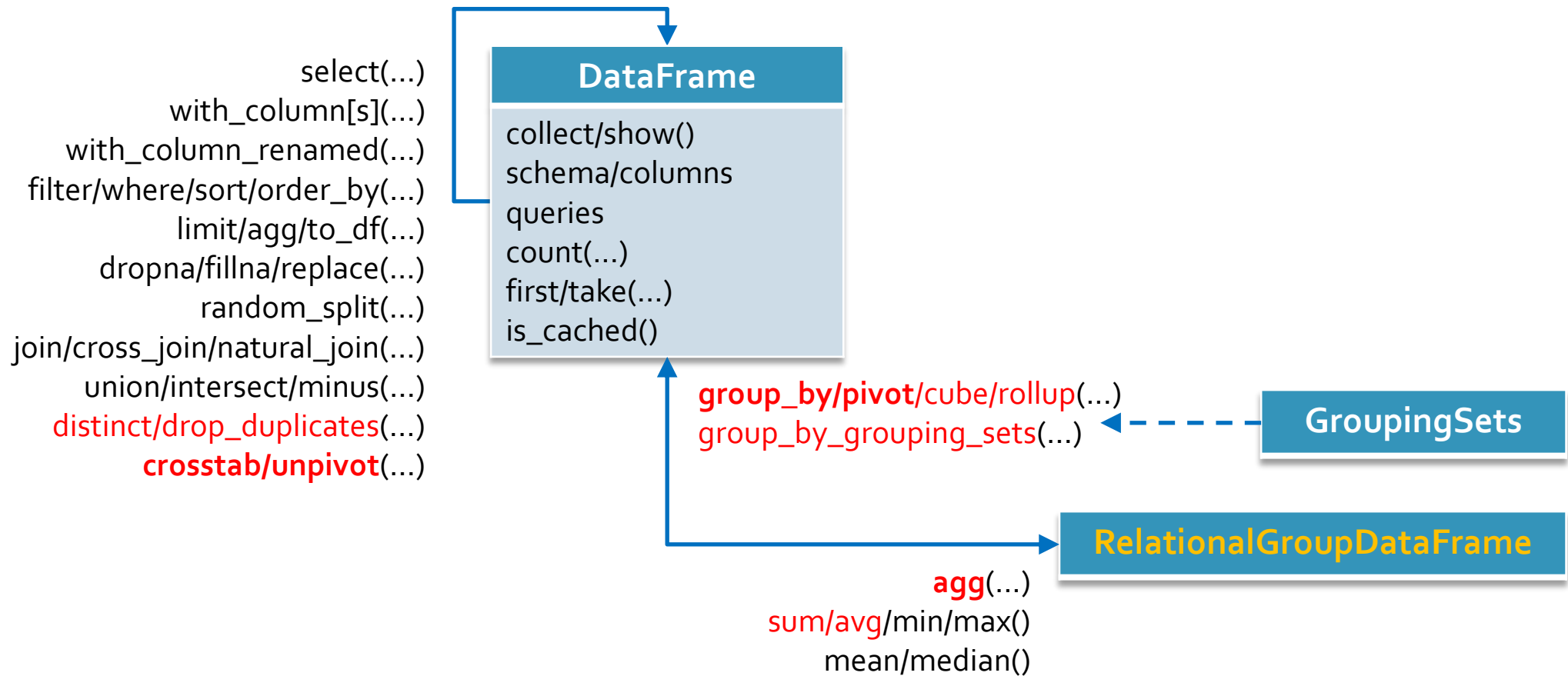
3.3 Perform aggregate and set-based operations on DataFrames

- Grouping



DataFrame Aggregations

- *distinct*(...) = ~*drop_duplicates*(...)
- *group_by*(...) → RelationalGroupDataFrame
- RelationalGroupDataFrame - w/ *agg*(...) + *sum/avg/min/max/mean/median*()
- *group_by_grouping_sets*(GroupingSets[]) → RelationalGroupDataFrame
- *cube/rollup*(...) → RelationalGroupDataFrame
- *pivot*(...) → RelationalGroupDataFrame
- *crosstab/unpivot*(...) → DataFrame



DataFrame Aggregations

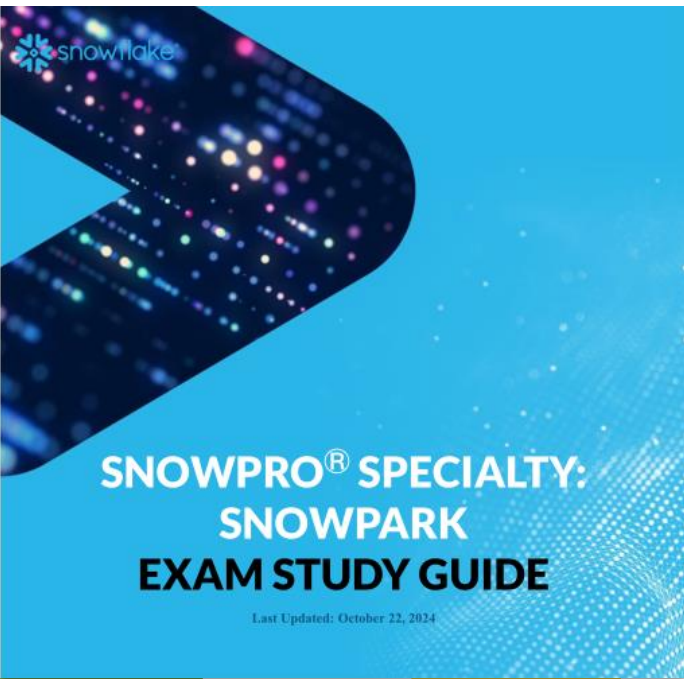


Window Functions in Snowpark

What We'll Cover (from the Study Guide)

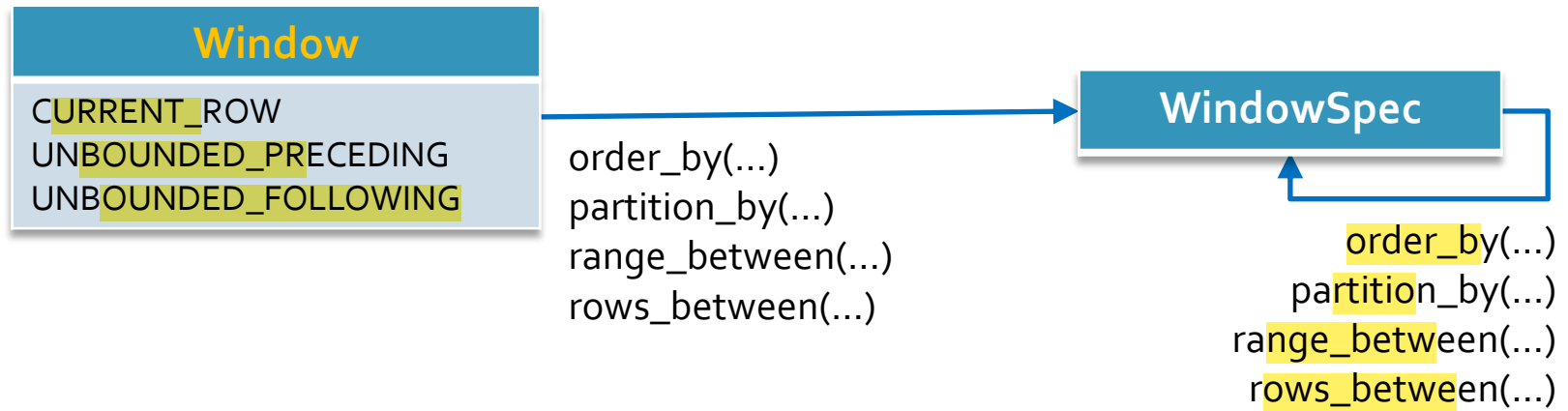
3.3 Perform aggregate and set-based operations on DataFrames

- Window



Window Functions in Snowpark

- `winFunc(...).over(winSpec)` → win func call in Snowpark (from **Column**)
- **Window** = (~static) class to *start* a **WindowSpec** obj, w/ ROWS/RANGE constants:
 - **CURRENT_ROW**
 - **UNBOUNDED_PRECEDING**
 - **UNBOUNDED_FOLLOWING**
- **WindowSpec** = class to (fully) build a *window specification*
 - `.partition/order_by(...)` → PARTITION/ORDER BY clauses
 - `.rows/range_between(start, end)` → ROWS/RANGE BETWEEN clauses



Window Function Classes

- Snowpark vs Pandas DataFrames
- Create Snowpark DataFrames
- Persist the Results of Snowpark DataFrames
- DML Operations using Snowpark DataFrames
- Process Semi-Structured Data from DataFrames

Snowpark DataFrame Integrations

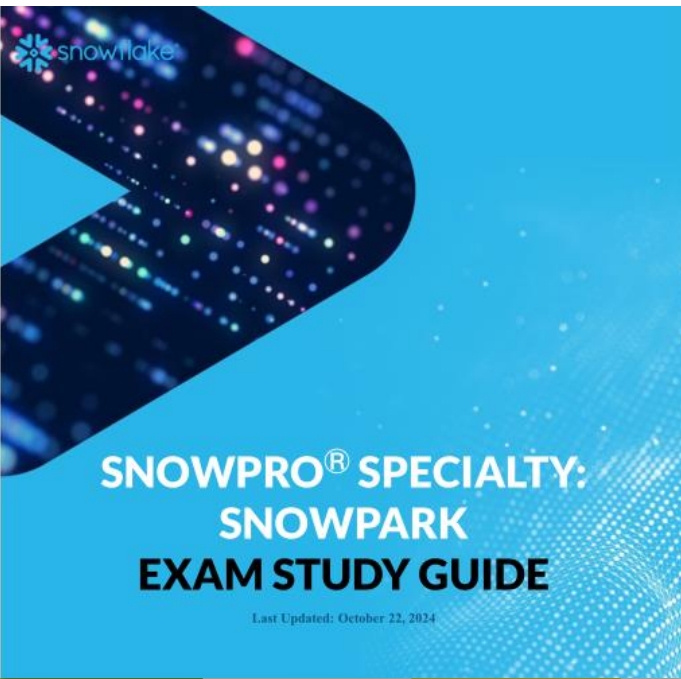


Snowpark vs Pandas DataFrames

What We'll Cover (from the Study Guide)

4.2 Enhance performance in Snowpark applications

- Vectorization
 - Snowpark DataFrames versus pandas on Snowflake



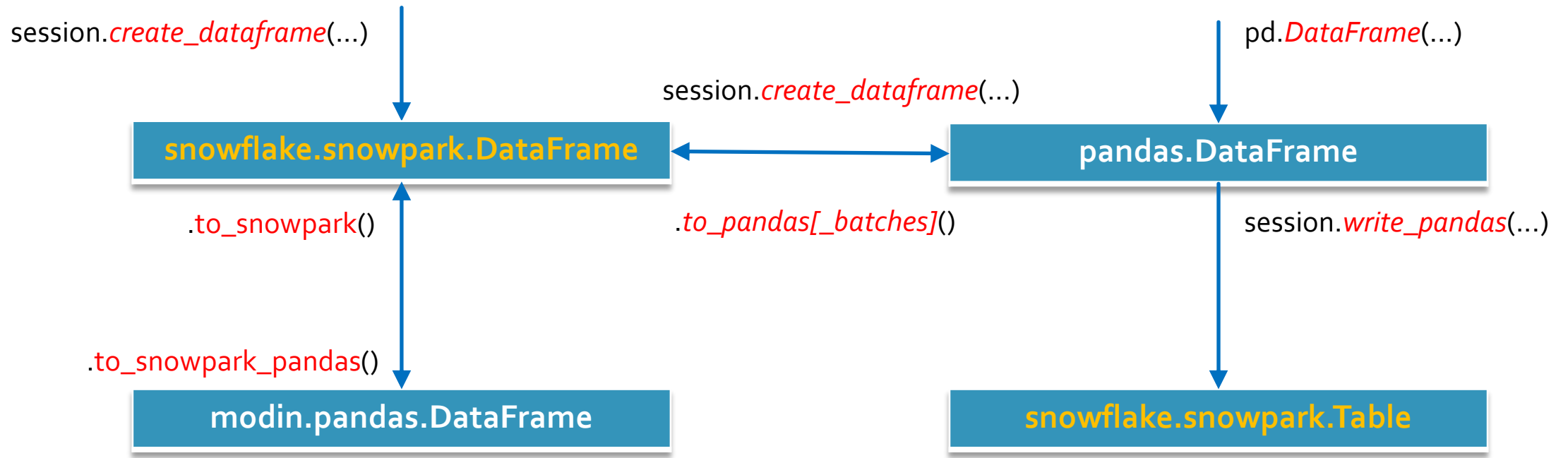
Pandas vs Snowpark DataFrame

Pandas DataFrame

- universal popular Python library
- **in-memory** data copy (on the client machine)
- **all processing is in-memory** (no SQL!)
- **may require a lot of memory**
- used also for I/O in Vectorized UDFs

Snowpark DataFrame

- leaves data in the database (for Snowflake only)
- **lazy evaluation** (~PySpark data frames)
- **generates/execs SQL query** on "action" methods
- **keeps only a pointer** to stored table data
- may still require a Snowpark-optimized warehouse



Pandas+Snowpark DataFrames

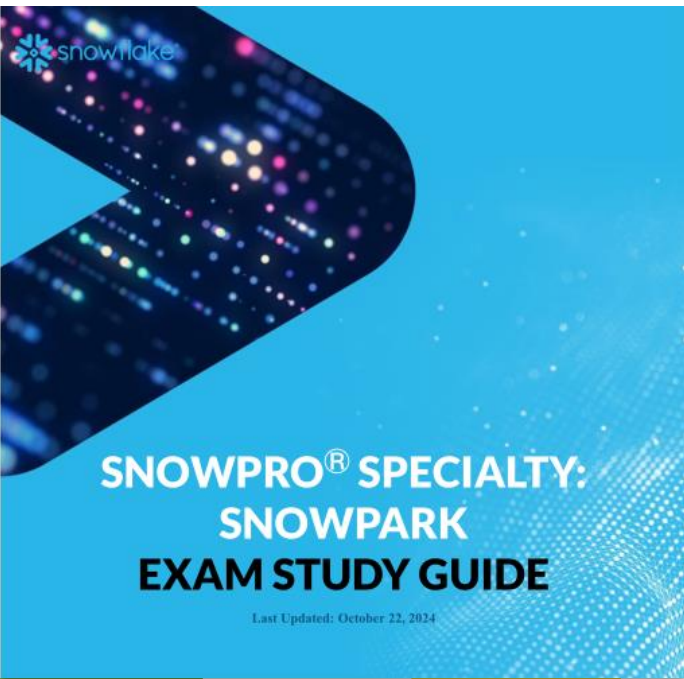


Create Snowpark DataFrames

What We'll Cover (from the Study Guide)

2.3 Create Snowpark DataFrames

- Multiple methods to create Snowpark DataFrames
 - From Snowflake tables/views
 - From Python objects (list, dictionary)
 - From SQL statements
 - From files (JSON, CSV, Parquet, XML)
 - From Pandas DataFrames
- Schemas (apply to DataFrames)
- Data types (for example, IntegerType, StringType, DateType)



Create DataFrame

- `session.table(name)` ← from table/view/stream
- `session.sql("SELECT ...")` ← from SQL query result
- `session.range(from, to, increment)` ← from range of numeric values
- `session.read.schema(...).csv("@stage/file.csv")` ← from file (through reader)
- `session.create_dataframe([...])` ← from 1+ rows, w/ def cols
 - `session.create_dataframe([Row(c1=..., ...), ...])` ← from array of Row objs
 - `session.create_dataframe([[...], ...], schema=[...])` ← 1+ rows, w/ col defs (~`.to_df(...)`)
 - `session.create_dataframe(df_pandas)` ← from pandas DataFrame (or pandas in Snowflake!)



Persist the Results of Snowpark DataFrames

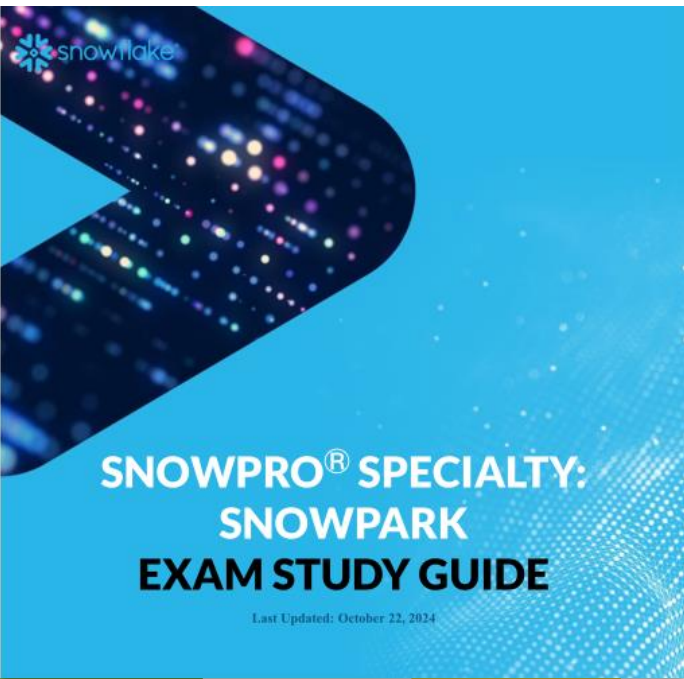
What We'll Cover (from the Study Guide)

3.5 Persist the results of Snowpark DataFrames

- Create views from DataFrames
- Save DataFrame results as Snowflake tables
- Save DataFrame results as files in a stage

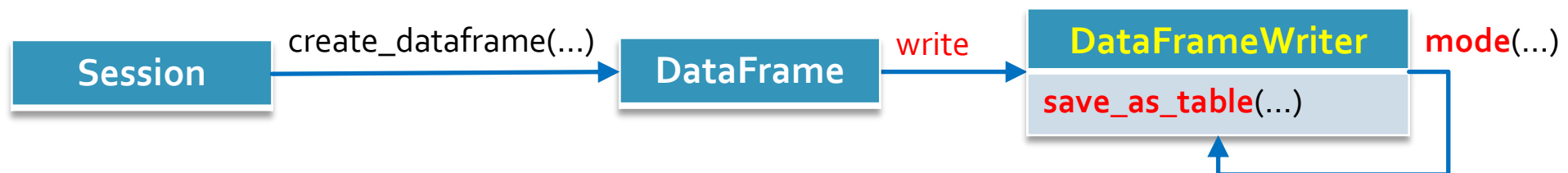
4.2 Enhance performance in Snowpark applications

- Materialize results (caching)
 - Caching DataFrames (using `.cache_result()`) and understanding why this is useful
 - Create a temporary table



Writing DataFrame data to a Table

1. Create a DataFrame w/ data to be written ← `df = create_dataframe(...)`
2. Get the DataFrameWriter object ← `writer = df.write`
3. Specify the save mode ← `writer = writer.mode(save_mode)`
 - “*append*” = adds data from DataFrame to existing table + creates new table if not there
 - “*overwrite*” = replace all existing table data
 - “*errorifexists*” = throws an exception if the table already exists
 - “*ignore*” = quietly skips the whole operation if the table already exists
4. Save DataFrame content into a table ← `writer.save_as_table(table_name)`



Persist DataFrame

- `df.write.mode("overwrite").save_as_table("name")` → append/replace temp/table
- `df.create_or_replace_temp_view("name")` → view or dynamic table
- `df.write.copy_into_location("@stage/file.csv", ...)` → ~COPY INTO file (from DF)
- `df.copy_into_table("name", ...)` → ~COPY INTO table
- `df.cache_result()` → into temp table (not affected by changes on the source df!)
 - `df.drop_table()` → clear cache (or use: `with df.cache_result() as df2: ...`)
 - `df.is_cached`

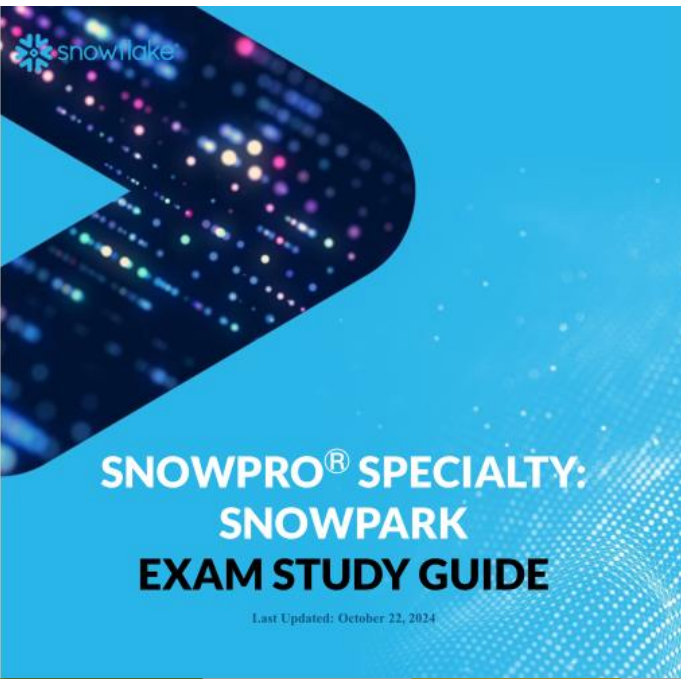


DML Operations using Snowpark DataFrames

What We'll Cover (from the Study Guide)

3.6 Perform DML operations using Snowpark DataFrames

- Delete data
- Update data
- Insert data
- Merge data



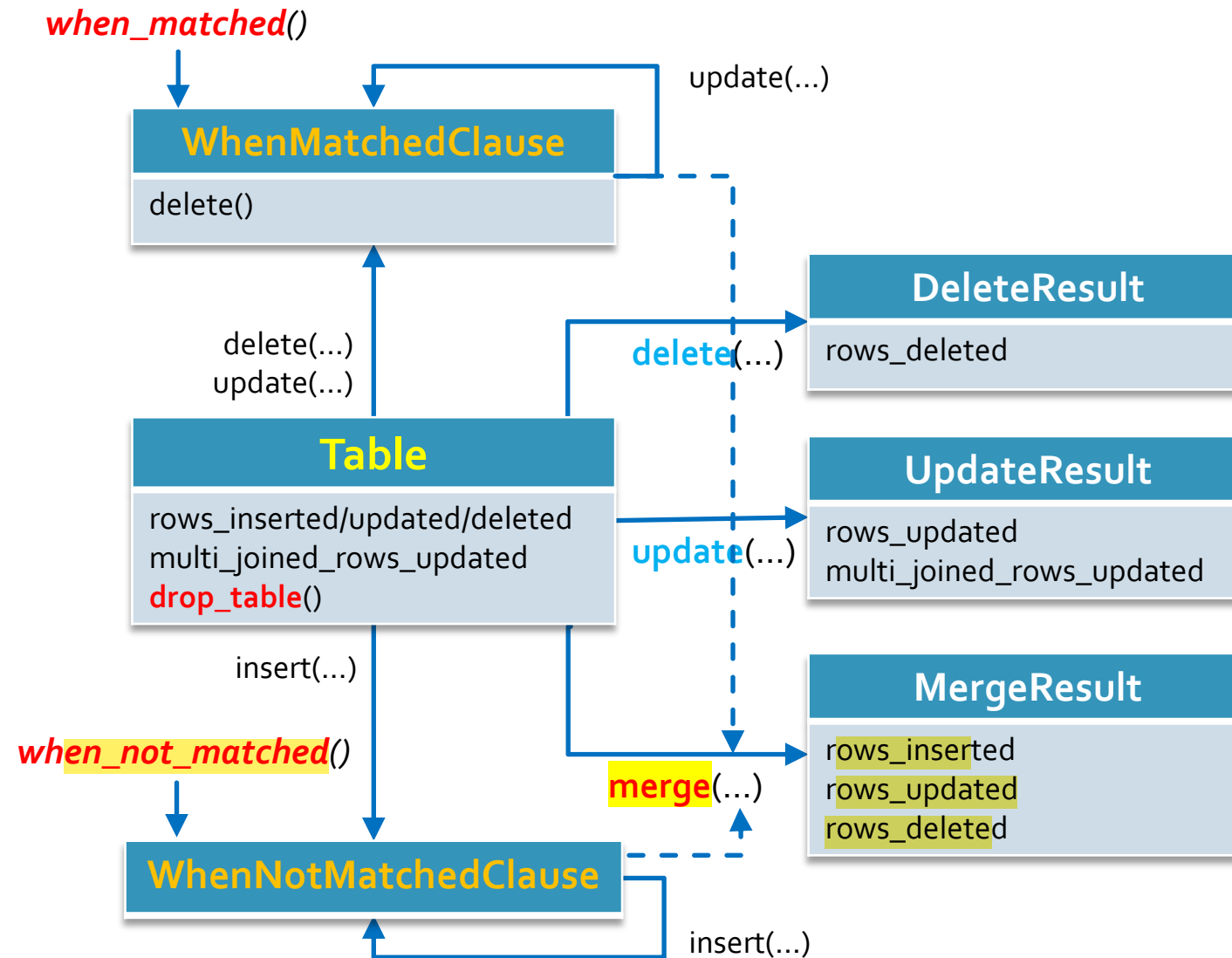


Table Class + Merge Function



Process Semi-Structured Data from DataFrames

What We'll Cover (from the Study Guide)

2.3 Create Snowpark DataFrames

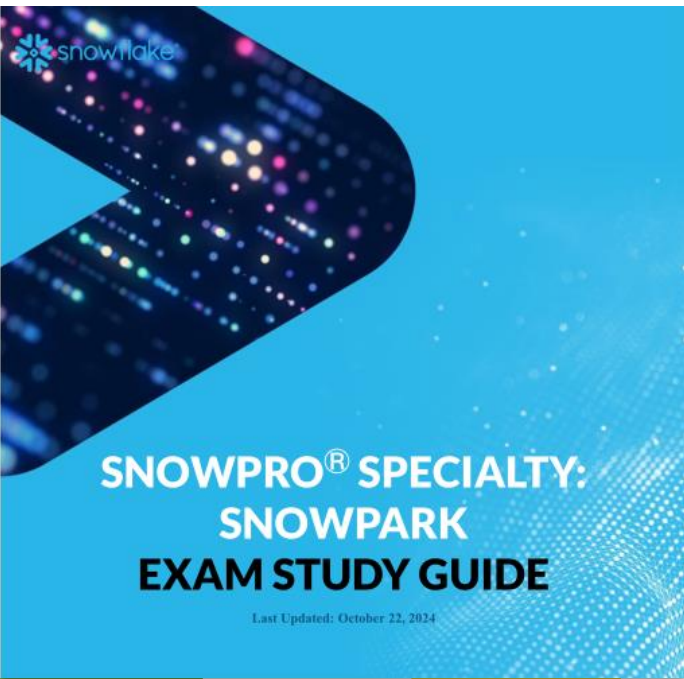
- From files (JSON, XML)

3.4 Transform semi-structured data in DataFrames

- Traverse semi-structured data
- Explicitly cast values in semi-structured data
- Flatten an array of objects into rows
- Load semi-structured data into DataFrames

3.2 Clean and enrich data using Snowpark for Python

- Perform joins



JSON SQL vs Snowpark DataFrame

- select `$1:...` from `@...` (`file_format => ...`)
- select `src:...` from `,,`
- select `src:dealership::string`
- select `src:customer[0].name::string`
- select `src[customer][0]['name']::string`
- select `get(src:vehicle[0], 'make')`
- select `get_path(src:vehicle, '[0]')`
- select `*` from `t`, `lateral flatten(src:vehicle[0])`
- `session.read.json(@...).select(sql_expr("$1:..."))`
- `session.table(...).select(col("src")...)`
- `.select(col("src")["dealership"].cast(StringType()))`
- `.select(sql_expr("src:customer[0].name::string"))`
- `.select(col("customer")[0]['name'].cast(StringType()))`
- `.select(get(col("src")["vehicle"][0], lit('make')))`
- `.select(get_path(col("src"), lit("vehicle[0]")))`
- `.join_table_function("flatten", col("src")["vehicle"][0])`

JSON from Snowpark DataFrame

- all JSON data is usually loaded into one single VARIANT table column
- `session.read.json(...)` → loads data from a staged JSON file into a DataFrame
- direct staged JSON file access → w/ `$1 + JSON (dot) path expression`
- `src:prop1[index].prop2::type ~= src['prop1'][index]['prop2']::type`
- `::string` (SQL cast) → `.cast(StringType())`
- traversing JSON data → w/ dot path expression (but not in Python DataFrame!)
- `sql_expr` → embedded SQL
- GET/GET_PATH functions → shortcuts for the dot notation
- LATERAL FLATTEN (from SQL) → `join_table_function("flatten", ...)`

- Create Procedures and Functions in SQL
- Call Procedures and Functions in Snowpark
- Python Worksheets
- Create Stored Procedures in Snowpark
- Create User-Defined Functions in Snowpark
- Vectorized User-Defined Functions
- Secure Stored Procedures and Functions
- Stages and File Operations
- Packages and Imports
- Run Stored Procedures in Snowpark



Procedures and Functions in Snowpark

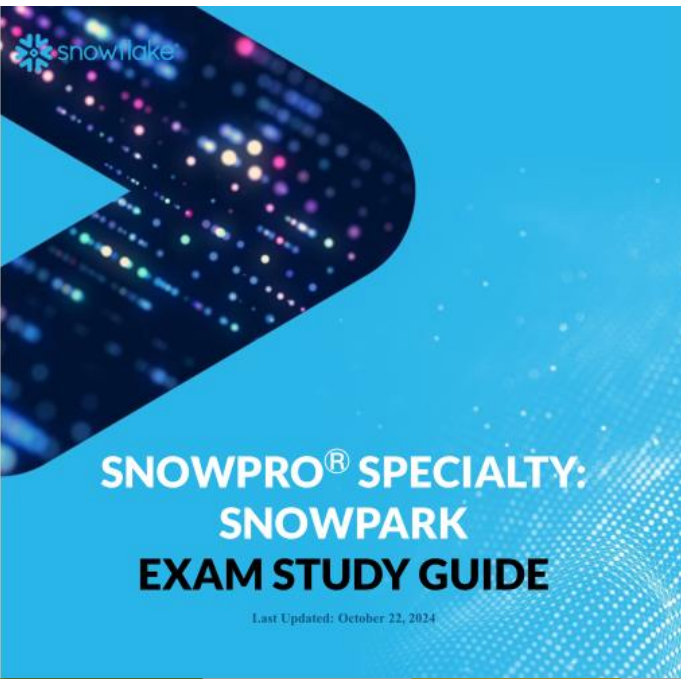


Create Procedures and Functions in SQL

What We'll Cover (from the Study Guide)

1.1 Outline Snowpark architecture

- Use of key objects
 - User-Defined Functions (UDFs)
 - User-Defined Table Functions (UDTFs)
 - Stored procedures



Stored Procedures (SPs) and User-Defined Functions (UDFs)

stored procedures (SPs) → can run SQL stmts and return a value

`CALL proc(...)` → cannot pass ret value into a var (unless TABLE)

user-defined functions (UDFs) → have more limitations

`SELECT func(...)` → return a scalar value

user-defined table funcs (UDTFs) → return a TABLE(...) as data type

`SELECT ... FROM TABLE(funcT(...))` → return cols w/ rows

Implementation Details

SQL/Snowflake Scripting

- optional "language sql"
- AS 'SELECT ...'; / AS \$\$ SELECT ... \$\$; / AS BEGIN .. END; (for SPs)

JavaScript

- w/ optional STRICT keyword, AS \$\$... \$\$;
- arg names must be always used upper case in the def block (ex: num → NUM)
- specific heavy implementation for UDTFs (w/ processRow)

Python

- stored procs always w/ Snowpark (first handler param is a Snowpark session!)
- HANDLER required (w/ fct name / MyClass w/o process for UDTFs)

CREATE PROCEDURE/FUNCTION

- **RETURNS** type [NOT NULL] / TABLE [(col type, ...)] → required, UDTF if TABLE
- **LANGUAGE PYTHON** → required
- **RUNTIME_VERSION** = 3.9/3.10/3.11 → required (check in PACKAGES view)
- **PACKAGES** = ('snowflake-snowpark-python[==0.1.1]', ...) → req Snowpark for SP
- **IMPORTS** = ('@stage/file', ...) → ext deps (from stage!)
- **HANDLER** = 'fct_name' → required Python entry point def function (module prefix!)
- **EXTERNAL_ACCESS_INTEGRATIONS** = ('...', ...) → to access external REST APIs
- **SECRETS** = ('var' = name, ...) → from internal vault
- **CALLED ON NULL INPUT** / **RETURNS NULL ON NULL INPUT** (or STRICT)
- **EXECUTE AS OWNER / CALLER** → important!
- **VOLATILE/IMMUTABLE** → deprecated

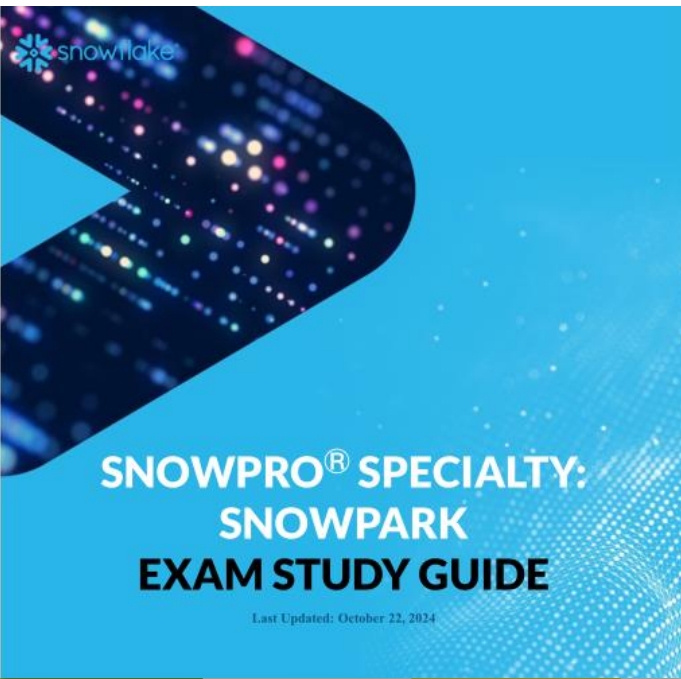


Call Procedures and Functions in Snowpark

What We'll Cover (from the Study Guide)

3.3 Perform aggregate and set-based operations on DataFrames

- Functions
- Table functions
- UDFs



Call Procedures and Functions

- `snowflake.snowpark.functions` → namespace w/ most built-in func wrappers
- `df.select(f(...))` → Snowpark wrapper to built-in SQL func
- `df.select(function("f")(...))` → explicit wrapper (fct obj) to built-in SQL func
- `df.select(call_function/builtin("f", ...))` → directly built-in SQL func
- `session.call("sp", ...)` → own registered stored proc, by name, w/ params
- `df.select(call_udf("f", ...))` → UDF, by name, w/ params
- `session.table_function("f", ...)` → UDTF or built-in SQL table func
- `session.table_function(call_table_function("f", ...))` → built-in SQL table func
- `df.join_table_function("f", ...)` → UDTF or built-in table func on DataFrame

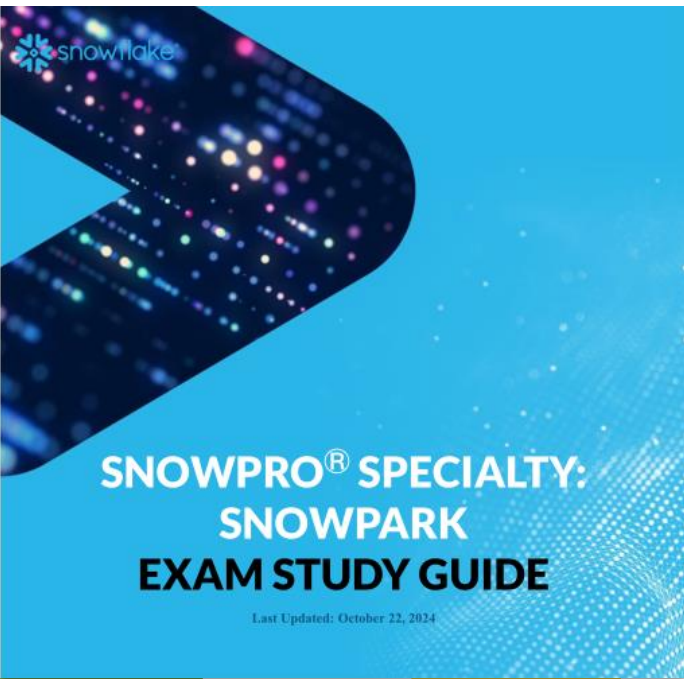


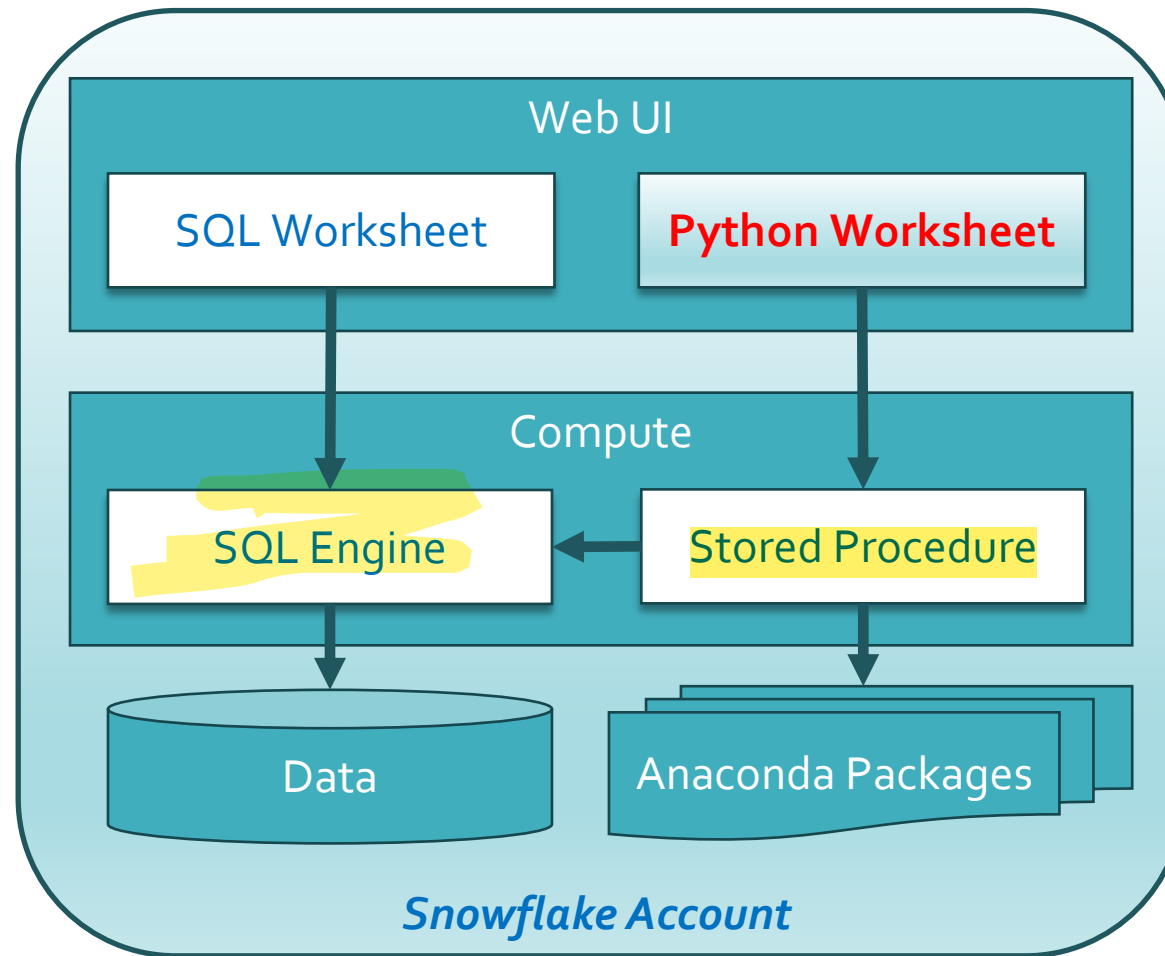
Python Worksheets

What We'll Cover (from the Study Guide)

1.1 Outline Snowpark architecture

- Use of key objects
 - Stored procedures





Python vs SQL Worksheet

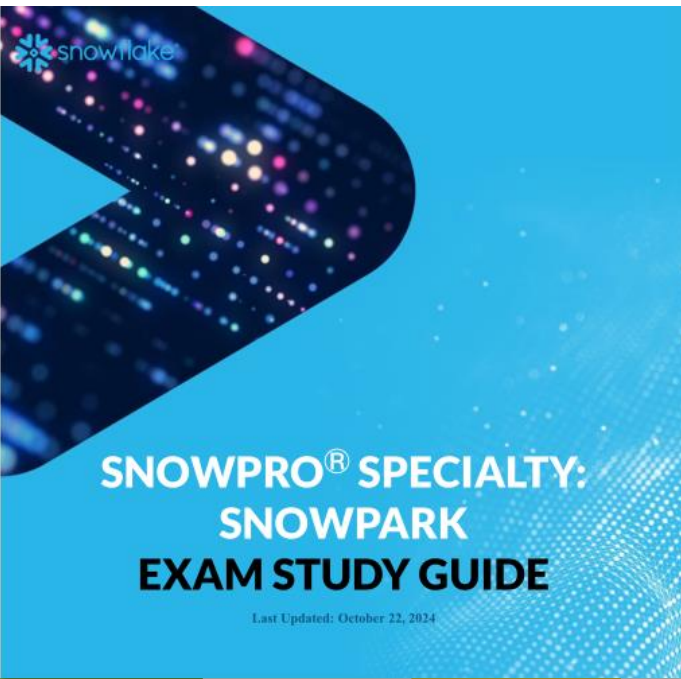


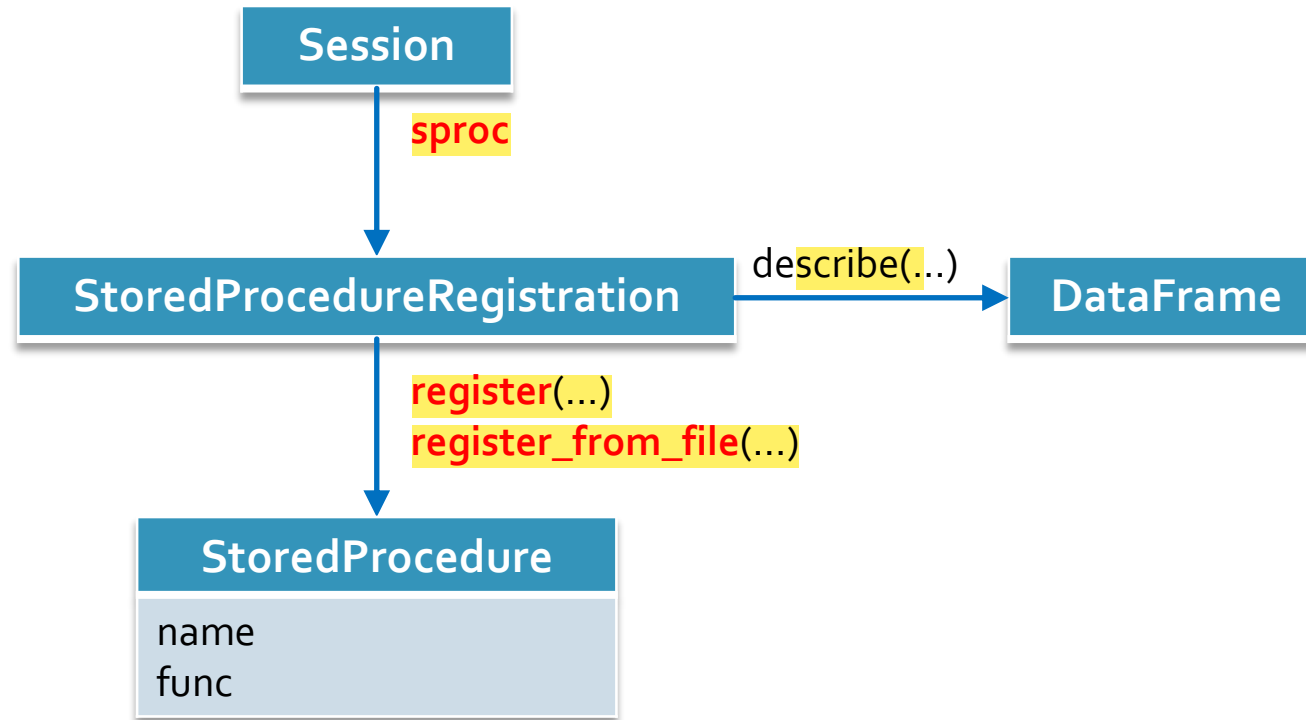
Create Stored Procedures in Snowpark

What We'll Cover (from the Study Guide)

2.5 Operationalize Snowpark stored procedures

- Create stored procedures from files (locally, on stage)
- Write Python functions to power stored procedures
- Register stored procedures
- Data types (type hints vs. registration API)
 - Provide the data types as parameters when creating a stored proc to return as Python hints/specify them as part of the registration





Procedures through Snowpark

Stored Procedures through Snowpark

- `f = sproc(lambda ..., [name=...])` ← `anonymous/named` (w/ inline lambda)
- `@sproc(is_permanent=True, stage_location= ...)` ← `registered` (w/ fct decorator)
- `session.sproc.register(...)` ← `registered/permanent` (alt to @sproc)
- `call` → `f(...)`, `session.call("f", ...)`, `session.sql("call f(...)")`
- `packages=["...", ...]` → from `session.add_packages/requirements("...", ...)`
- `replace=True, if_not_exists=True` → for overrides
- `input_types=[...], return_type=...` → req if no *type hints* (in Python → `x: int, ...`)
- `execute_as, strict, secrets, ...` → other params

Stored Procedures from Files

- `session.sproc.register_from_file(file_path="file.py", func_name=...)` ← from local Python/ZIP file (copied into a stage)
- `session.sproc.register_from_file(file_path="@stage/file.py", func_name=..., return_type=...)` ← from staged Python/ZIP file (ret type required!)
- `imports=["...", ...]` → from `session.add_import(...)`, for local/staged files
- *type hints* cannot be extracted from ZIP
- *cloudpickle* → to pack/serialize all code/deps into bytecode through stage

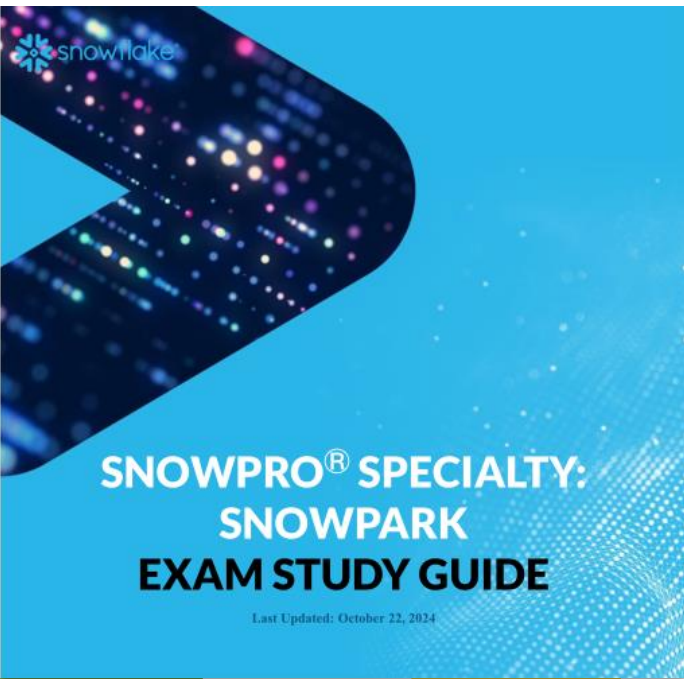


Create User-Defined Functions in Snowpark

What We'll Cover (from the Study Guide)

2.4 Operationalize UDFs and UDTFs in Snowpark

- Create UDFs from files (locally, on a stage)
- Write Python function to create UDFs and UDTFs
- Register UDFs and UDTFs (for example, `session.utf(...)`, `functions.utf(...)`)
- Data types (type hints vs. registration API)
 - Provide the data types as parameters when creating a UDF or UDTF to return as Python hints/specify them as part of the registration





UDFs through Snowpark

UDFs through Snowpark

- `f = udf(lambda ..., [name=...])` $\leftarrow$ *anonymous/named* (w/ inline lambda)
- `@udf(is_permanent=True, stage_location= ...)` $\leftarrow$ *registered* (w/ fct decorator)
- `session.udf.register(...)` $\leftarrow$ *registered/permanent* (alt to @udf)
- *call* $\rightarrow$ `f(...)`, `call_udf("f", ...)`, `session.sql("select f(...)")`
- `packages=["...", ...]` $\rightarrow$ from `session.add_packages/requirements("...", ...)`
- `replace=True, if_not_exists=True` $\rightarrow$ for overrides
- `input_types=[...], return_type=...` $\rightarrow$ req if no *type hints* (in Python $\rightarrow$ `x: int, ...`)
- `strict, secrets, ...` $\rightarrow$ other params (no **execute as owner/caller!**)

UDFs from Files

- `session.udf.register_from_file(file_path="file.py", func_name=...)` ← from local Python/ZIP file (copied into a stage)
- `session.udf.register_from_file(file_path="@stage/file.py", func_name=..., return_type=...)` ← from staged Python/ZIP file (ret type required!)
- `imports=["...", ...]` → from `session.add_import("...")`, for local/staged files
- *type hints* cannot be extracted from ZIP
- *cloudpickle* → to pack/serialize all code/deps into bytecode through stage



Vectorized User-Defined Functions

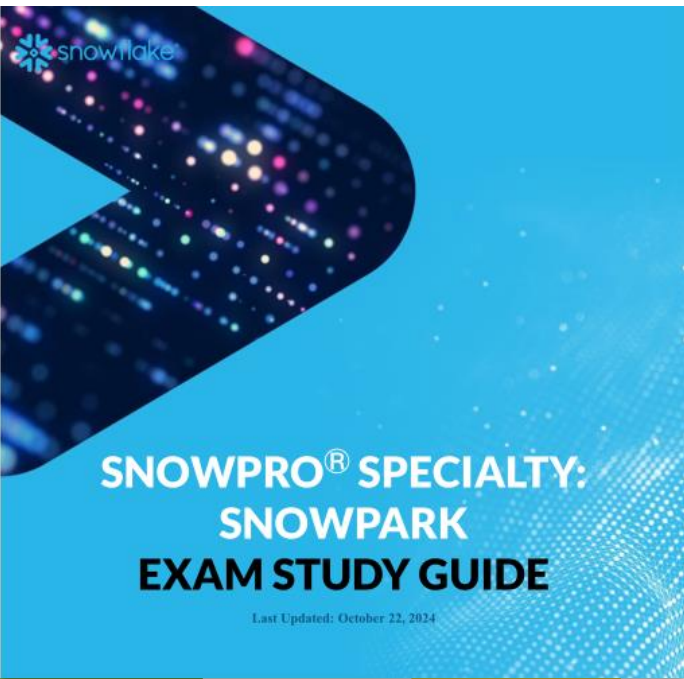
What We'll Cover (from the Study Guide)

2.4 Operationalize UDFs and UDTFs in Snowpark

- Compare scalar and vectorized operations

4.2 Enhance performance in Snowpark applications

- Vectorization
 - Understanding the diff between vectorized and scalar UDFs
 - Vectorized UDFs for batching



Create Vectorized Python UDFs

in SQL

- `_snowflake` → internal package ref (must be imported)
- `@vectorized` function decorator
- *alt:* `f._sf_vectorized_input = pandas.DataFrame`
- `max_batch_size` → optional param, to limit target batch size (by def None)
- *alt:* `f._sf_max_batch_size = 5`

in Snowpark

- `pandas_udf` → Snowpark function, for anonymous vectorized UDF (w/ lambda)
- `@pandas_udf` → for vectorized named UDF (used as function!)
- `input_types=[...], return_type=...` → when no *type hints*

Vectorized Python UDF Key Points

- = w/ auto-buffers for the I/O parameters and (scalar) results
 - *input* → batches of rows as **Pandas DataFrames**
 - *output* → batches of results as **Pandas arrays / Series**
- could get better performance than the row-by-row operations
- call them the same way, as regular UDFs



Secure Stored Procedures and Functions

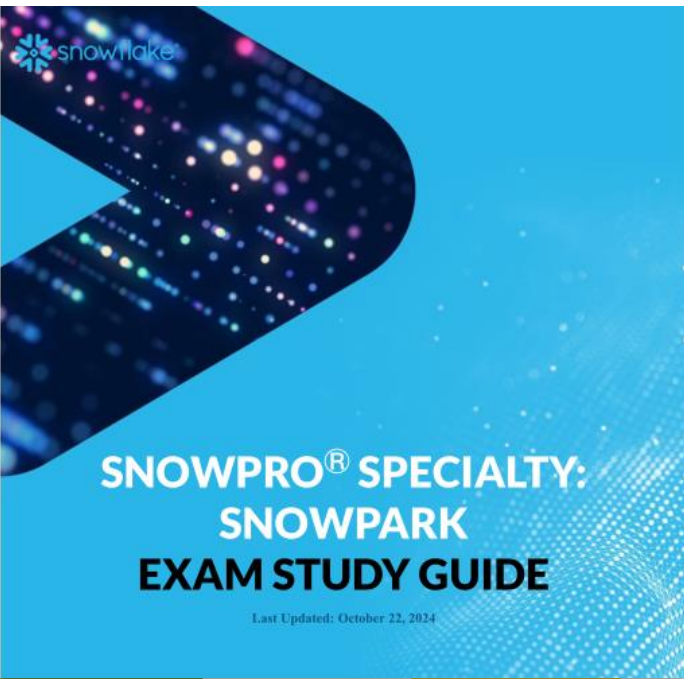
What We'll Cover (from the Study Guide)

2.5 Operationalize Snowpark stored procedures

- Secure stored procedures
 - Use SQL to alter stored procedures created with Snowpark
 - Caller versus owner rights

2.4 Operationalize UDFs and UDTFs in Snowpark

- Secure UDFs and UDTFs
 - Use SQL to alter UDFs and UDTFs created with Snowpark
 - Grant access to UDFs and UDTFs to share code
 - Understanding how to grant object permissions so other Snowflake users can see and use the UDFs and UDTFs



Secure Procedures and Functions

- CREATE **SECURE** PROCEDURE/FUNCTION/VIEW ... → **IS_SECURE** column in SHOW ...
- ALTER FUNCTION/VIEW ... **SET/UNSET SECURE** (not for PROCEDURE!)
- *sensitive info is hidden* from users/callers → still visible only for authorized users
- hidden body (GET_DLL, on DESC/SHOW/...) + handler + imports + packages
- hidden data scanned + total data
- can still see signature, implementation language, staged files used
- **query optimization** (like pushdown filters) is bypassed → performance affected
- should hide sequence-generated IDs
- CURRENT_ROLE/USER from shared consumer returns NULL

Owner vs Caller Rights

EXECUTE AS OWNER

- by def + only for stored procs (not UDFs)
- caller can execute/describe the SP, but no implicit access to the objects inside
- caller must have USAGE privilege on the procedure/database/schema
- caller cannot view the body of the SP
- *no session variables allowed* inside the SP

EXECUTE AS CALLER

- caller must have access to all objects internal to the SP
- caller can define *external session variables*, accessed internally in the SP as well



Stages and File Operations

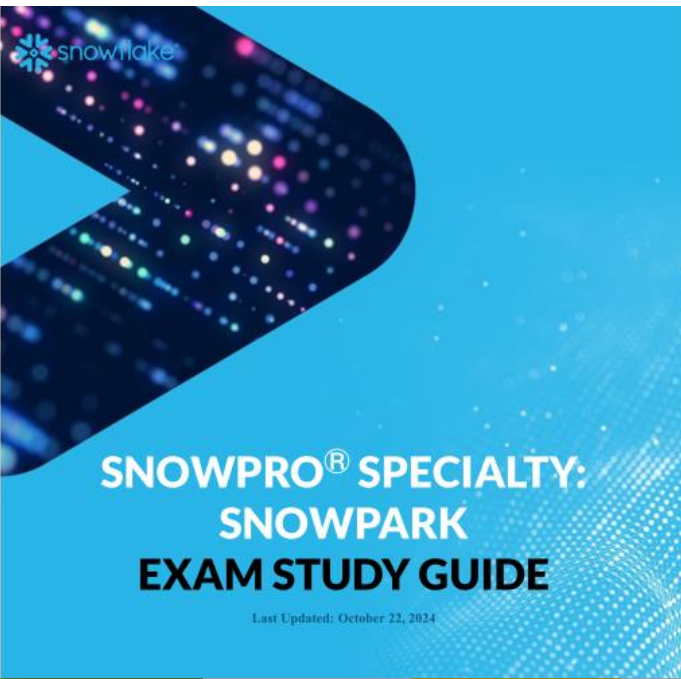
What We'll Cover (from the Study Guide)

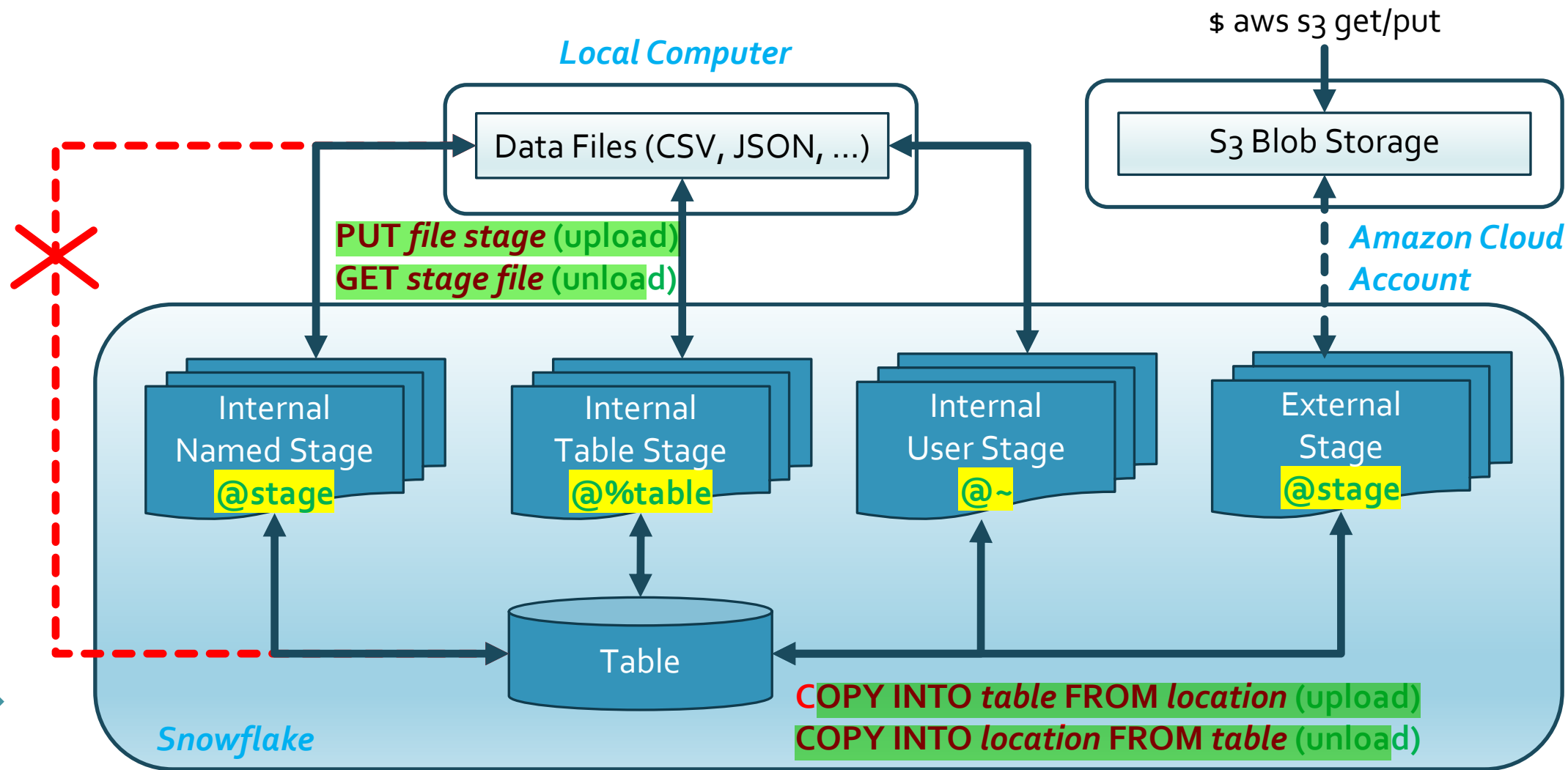
1.1 Outline Snowpark architecture

- Use of key objects
 - File operations

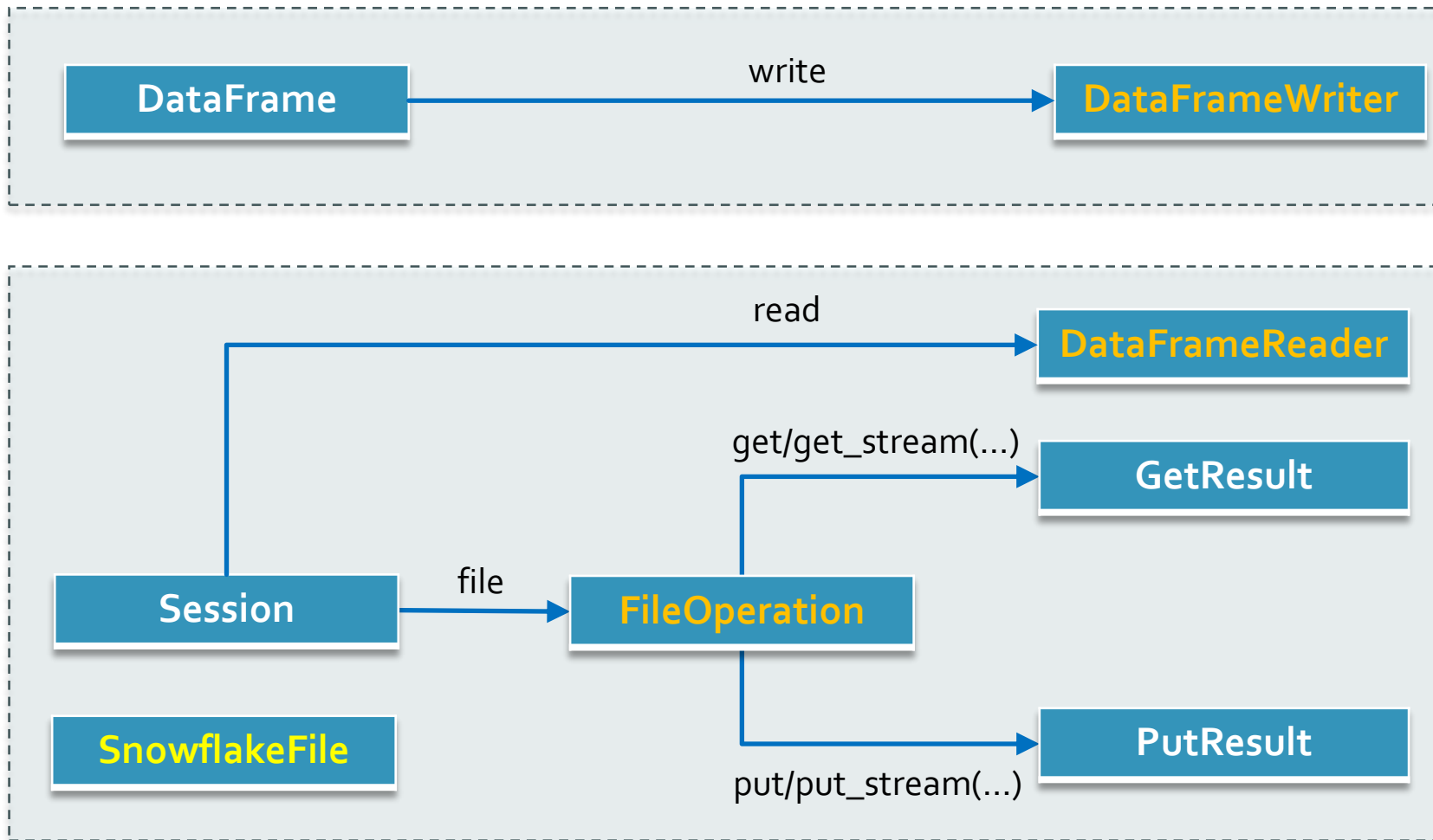
2.2 Use Snowpark with unstructured data

- Read files with SnowflakeFile object
- Use UDFs and UDTFs to process files
- Use stored procedures to process files





Snowflake Stages



File Operations

File Operations Key Points

- session.*file* → **FileOperation** → **Get/PutResult** (for GET/PUT commands)
- session.*read* → **DataFrameReader** → **CSV/JSON/PARQUET/XML/ORC/AVRO**
- df.*write* → **DataFrameWriter** → **CSV/JSON/PARQUET**
- **IMPORTS** → file copied in "snowflake_import_directory" + can use *open/read...*
- **SnowflakeFile** ← in *snowflake.snowpark.files*
 - to process unstructured data in UDFs/SPs
 - to stream files of any **size from stage** → w/ *open/close()*
 - *require_scoped_url*=False → to avoid scoped URLs (recommended against SQL injection)
 - *IOBase*: (*fileno/isatty/readable/...*) + *BufferedIOBase* (*readinto1* + *RawIOBase*: *read/readall*)



Packages and Imports

What We'll Cover (from the Study Guide)

1.1 Outline Snowpark architecture

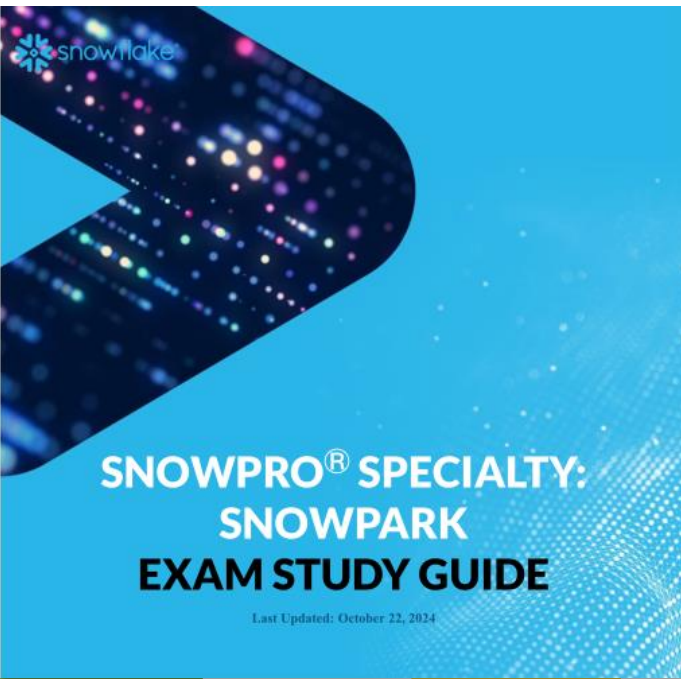
- Types of libraries (DataFrames, Machine Learning)
 - Anaconda repository (Python packages directly into Snowflake)
 - Other third-party libraries (not managed by Anaconda repository)

2.4 Operationalize UDFs and UDTFs in Snowpark

- Use Python modules (packaged Python code) with UDFs

2.5 Operationalize Snowpark stored procedures

- Use Python modules (packaged code, Anaconda) with stored procedures
- Make dependencies available to code
- Bring Python modules (packaged code) to be used with UDFs
 - Stored procedures to enable reuse of code

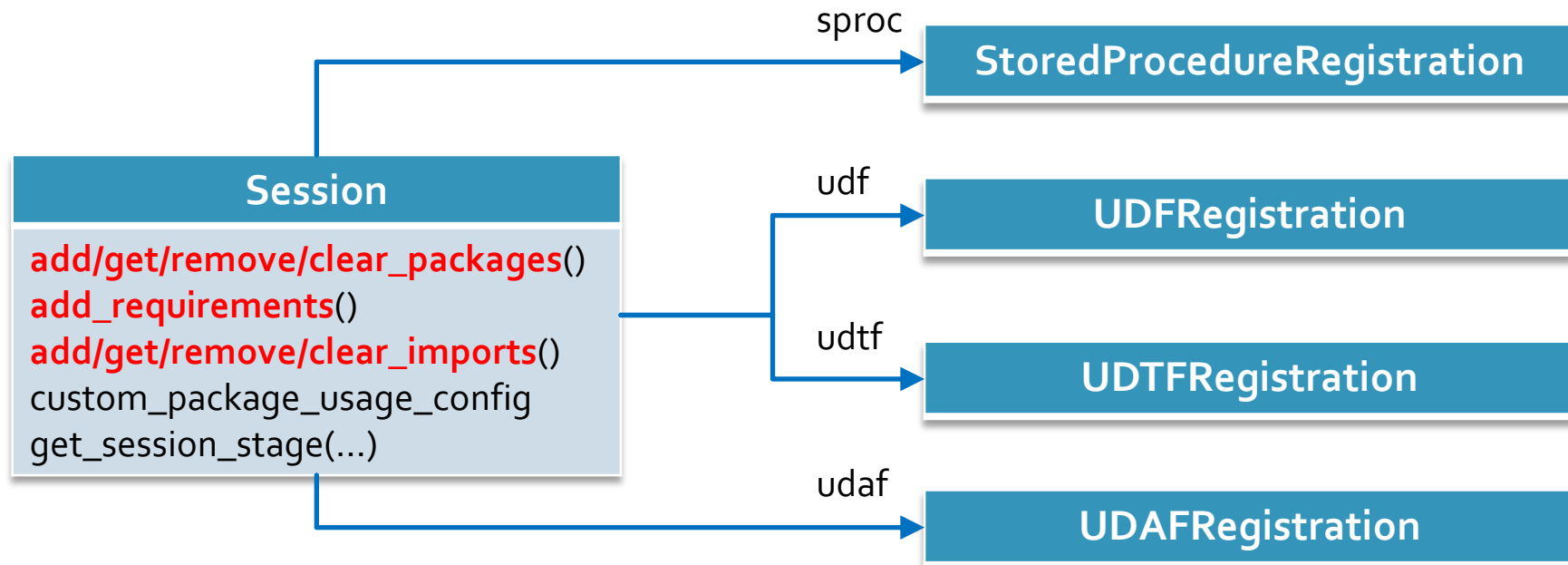


Published/Installed/Ref-ed Packages

- **PACKAGES** = ('snowflake-snowpark-python==0.1.13', ...) → referenced Anaconda packages, w/ their optional version (latest by def), in any SP/UDF definition
- **Anaconda Packages** = curated list of *published* (*installed* if *referenced*!) libs
- **snowflake-snowpark-python** = Snowpark package, always included in SPs
- **Session** param = first arg in any Snowflake SP **HANDLER** (in Python/Java/Scala)
- **Packages** in the web UI → it will be added to the PACKAGES list, if checked
- session.**add/remove/clean_packages**([...]) - manually manage ref-ed packages
- session.**add_requirements**("file") → requirements.txt (pip) / environment.yml (conda)
- **RUNTIME_VERSION** = 3.9 → Python runtime (avoid latest!), in any SP/UDF def
- INFORMATION_SCHEMA.**PACKAGES** → installed packages + versions + runtime

Imported External Dependencies

- **IMPORTS** = ('@stage/file.zip', ...) → ext packages/code modules/binary/CSV...
- all these staged files will be copied into a "snowflake_import_directory"
- could also pass file path as arg → as *scoped URLs* (to avoid SQL injection)
- only pure Python packages are allowed (Anaconda may have compiled C/C++)
- session.add/remove/clean_imports([...]) - manually manage ext dep files
- session.custom_package_usage_config = { "enabled": True } → to allow ext pack
- session.custom_package_usage_config = { "cache_path": ... } → to cache
- session.get_session_stage(...) → temp stage for uploading temp artifacts



Session with Packages&Imports

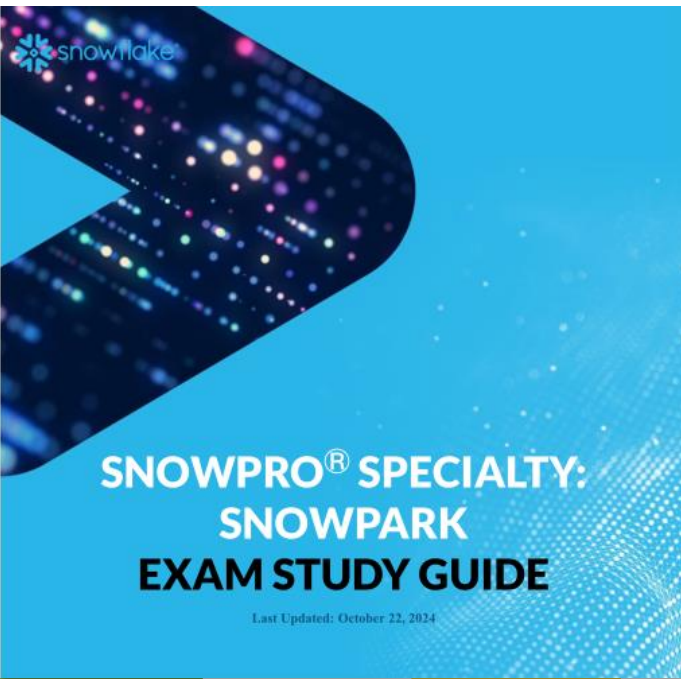


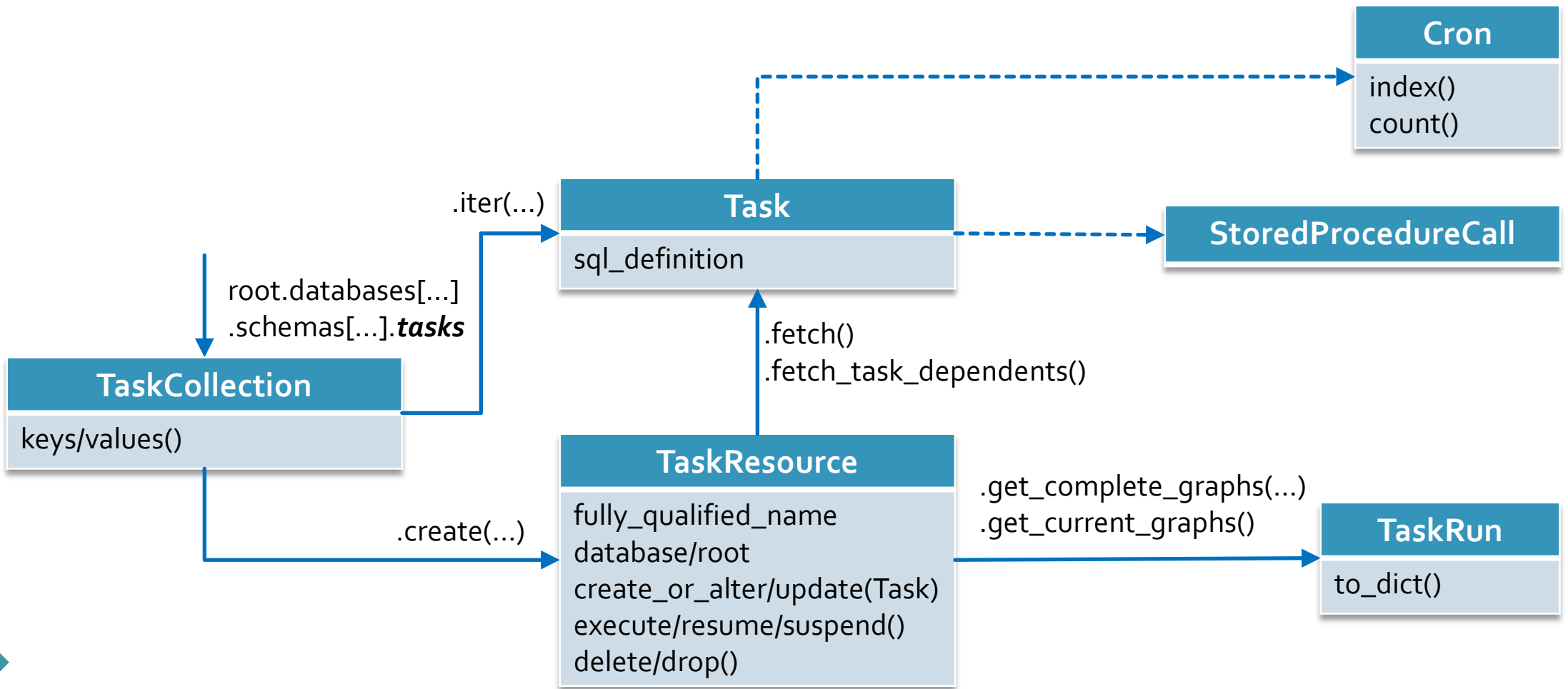
Run Stored Procedures in Snowpark

What We'll Cover (from the Study Guide)

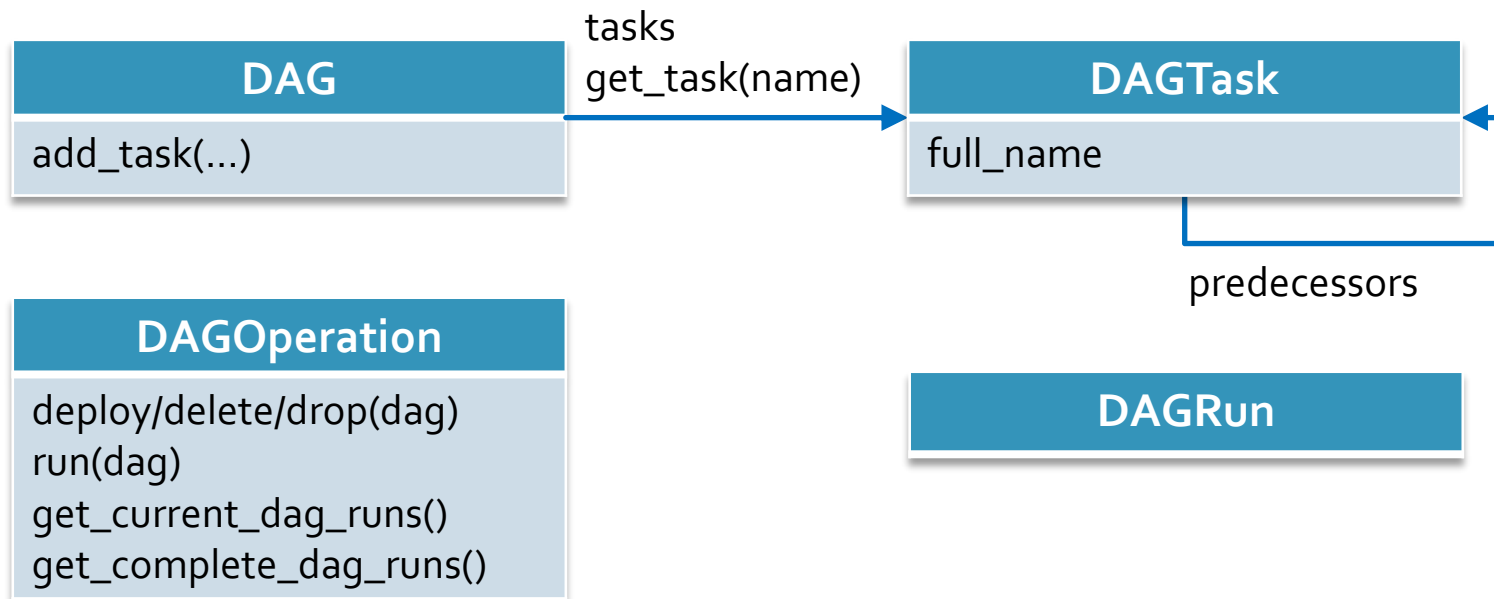
2.5 Operationalize Snowpark stored procedures

- Use Snowpark Python stored procedures to run workloads
- Create Directed Acyclic Graphs (tasks) executing stored procedures
 - Python API (Open Preview)





Task Python API



DAG Python API

DAG Tasks with Stored Procedures

- create a **DAG** → w/ *name* + *schedule* time (can use **Cron** obj!)
 - `snowflake.core.tasks.dagv1` → add `snowflake.core` to Packages!
 - `use_func_return_value=True` → w/ *stage_location* + packages
- create **DAGTask** objects → w/ **StoreProcedureCall** (or embedded SQL)
 - establish predecessors, w/ `tparent >> tchild` operator
- create **DAGTaskBranch** objects → for tasks w/ multiple successors
 - `task_branch >> [t1, t1]` → w/ *condition* on the return value!
 - `TaskContext(session).set_return_value/get_predecessor_return_value("name")`
- create **DAGOperation** `dag_op` → w/ `Root().databases["db"].schemas["sch"]`
 - `dag_op.deploy/run/get_current/complete_dag_runs("dag")`

- Snowpark-Optimized Warehouses
- Snowflake APIs
- Snowpark Sessions and Setup
- Observability in Snowpark
- Snowpark Test Environments
- Snowpark for Machine Learning



Snowpark Configuration and Environments

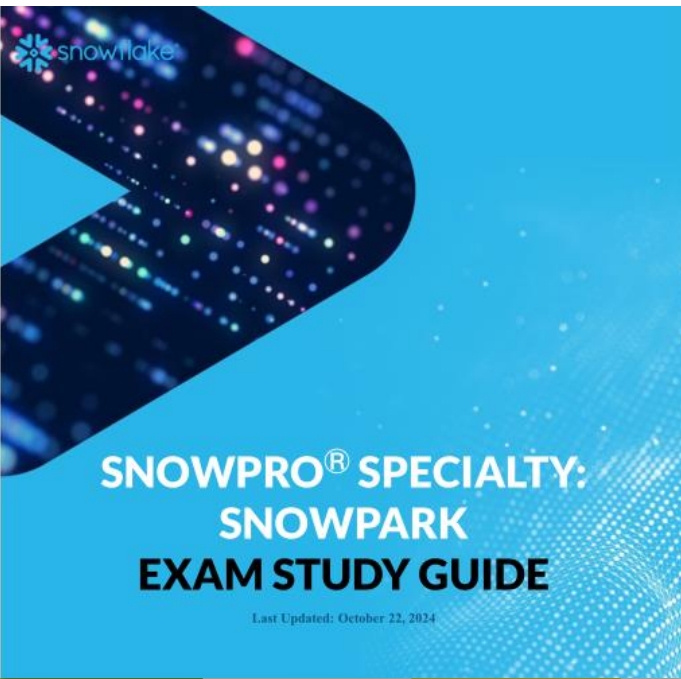


Snowpark- Optimized Warehouses

What We'll Cover (from the Study Guide)

4.1 Configure Snowpark-optimized warehouses

- Use cases for Snowpark-optimized virtual warehouses
- Modify Snowpark-optimized virtual warehouse props
- Billing for Snowpark-optimized virtual warehouses
- When to scale up/down virtual warehouses



Snowpark-Optimized Warehouses

- 16/64x RAM + 1.5x cost of Standard WHs
- *recommendations (use cases)*
 - for Model Training (not Model Serving!) in ML/DS → to reduce bytes spillage
 - for one stored proc deployed by Snowpark in a single-node WH
 - set MAX_CONCURRENCY_LEVEL = 1 → to maximize CPU usage + RAM resources
- *limitations*
 - from Medium+ (no X-Small/Small sizes) + no GPUs (see SPCS w/ NVIDIA)
 - create/resume may take longer + QAS not supported
- *scaling*
 - manual scale up/down, by size → for query performance (when complex queries)
 - automatic scale in/out, by multi-clusters → for concurrent multi-user access

CREATE/ALTER WAREHOUSE

- `WAREHOUSE_TYPE = 'SNOWPARK-OPTIMIZED'` → not `'STANDARD'` (suspend!)
- `WAREHOUSE_SIZE = 'MEDIUM'` (def for SO!) → no `'X-SMALL'` (~XSMALL)/`'SMALL'`
- `RESOURCE_CONSTRAINT = 'MEMORY_16X_X86'` → `'MEMORY_1/16/64X[_x86]'`
- `MAX_CONCURRENCY_LEVEL = 1` → for best CPU+RAM res utilization (by def 8)
- `AUTO_SUSPEND = ~60` → by def 10 minutes! (`AUTO_RESUME` is by def TRUE)
- `INITIALLY_SUSPENDED = TRUE` → by def FALSE!
- `STATEMENT_TIMEOUT_IN_SECONDS = 240` → by def 2 days!

Value	Memory (up to)	CPU architecture	Min warehouse size required	Max warehouse size
MEMORY_1X	16 GB	Standard	XSMALL	X4LARGE
MEMORY_1X_x86	16 GB	x86	XSMALL	X4LARGE
MEMORY_16X	256 GB	Standard	MEDIUM	X6LARGE
MEMORY_16X_x86	256 GB	x86	MEDIUM	X4LARGE
MEMORY_64X	1 TB ^[1]	Standard	LARGE	X4LARGE
MEMORY_64X_x86	1 TB ^[1]	x86	LARGE	X4LARGE

RESOURCE_CONSTRAINT

Type of Virtual Warehouse	X-Small	Small	Medium	Large	X-Large	2X-Large	3X-Large	4X-Large	5X-Large	6X-Large
Standard	1	2	4	8	16	32	64	128	256	512
Snowpark-optimized	n/a	n/a	6	12	24	48	96	192	384	768

Billing for Snowpark-Optimized

Credits per hour										
Resource Constraint	XS	S	M	L	XL	2XL	3XL	4XL	5XL	6XL
MEMORY_1X	1.00	2.00	4.00	8.00	16.00	32.00	64.00	128.00	•	•
MEMORY_1X_X86	1.10	2.20	4.40	8.80	17.60	35.20	70.40	140.80	•	•
MEMORY_16X	•	•	6.00	12.00	24.00	48.00	96.00	192.00	384.00	768.00
MEMORY_16X_X86	•	•	6.25	12.50	25.00	50.00	100.00	200.00	•	•
MEMORY_64X	•	•	•	15.00	30.00	60.00	120.00	240.00	•	•
MEMORY_64X_x86	•	•	•	16.00	32.00	64.00	128.00	256.00	•	•

Billing for Snowpark-Optimized

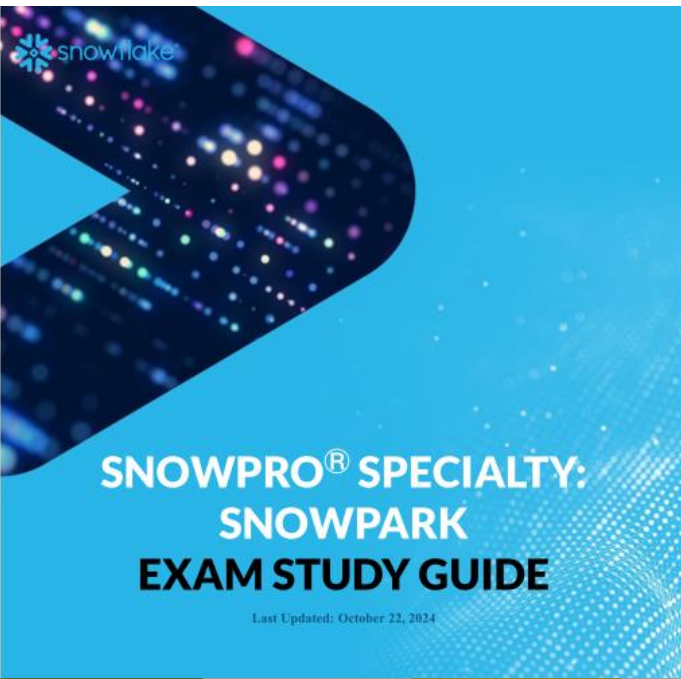


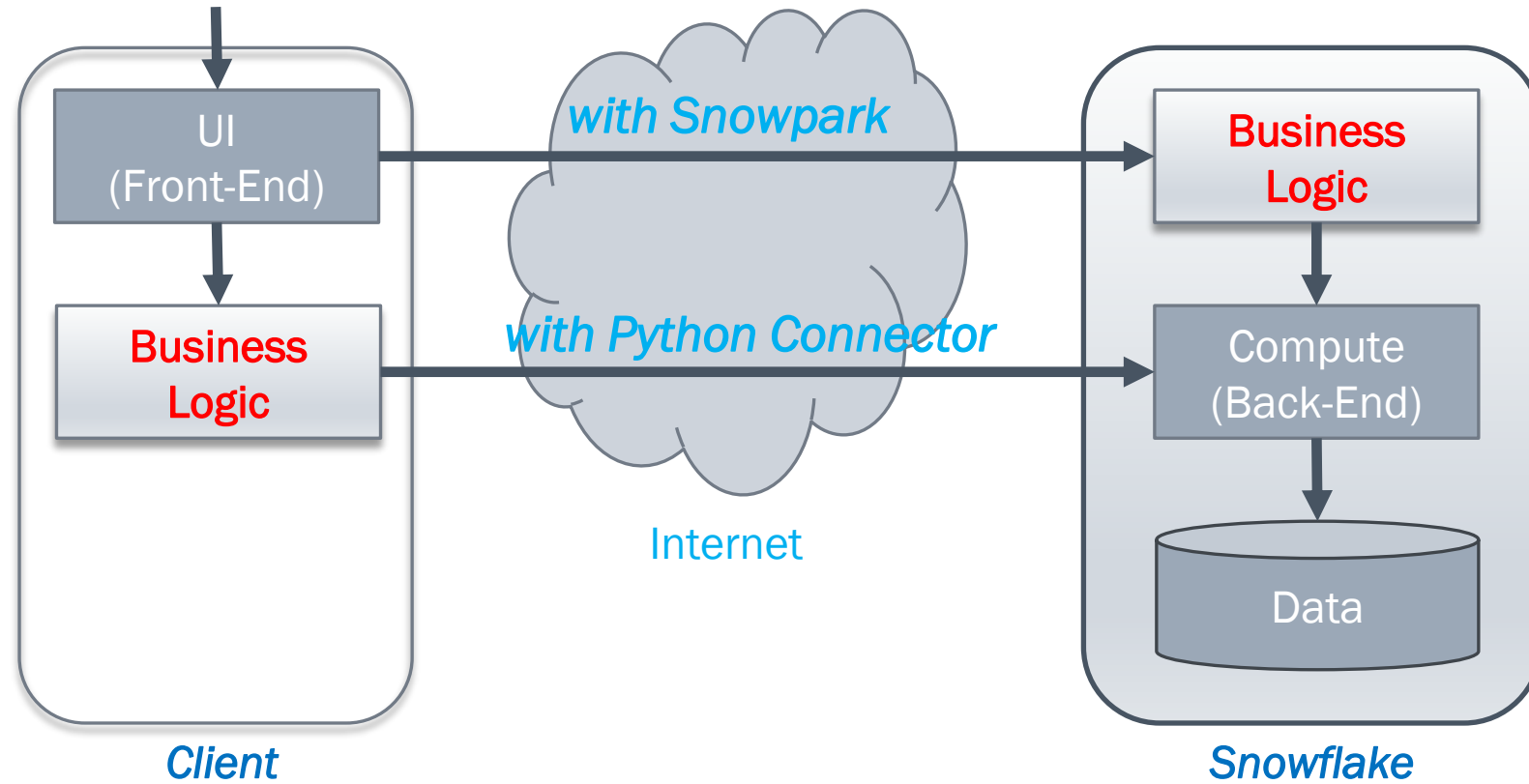
Snowflake APIs

What We'll Cover (from the Study Guide)

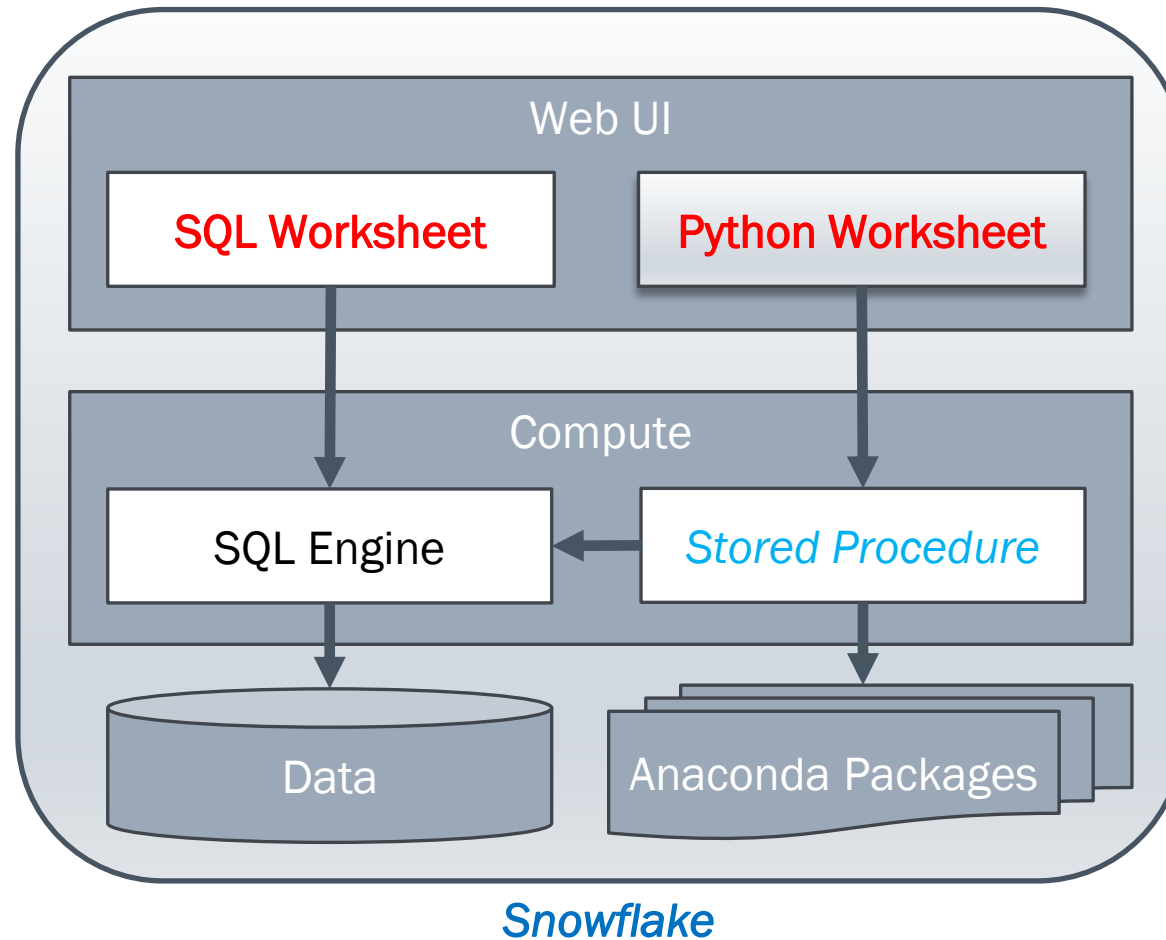
1.1 Outline Snowpark architecture

- Client-side and server-side capabilities





Snowpark vs Python Connector



SQL vs Python Worksheet

Snowpark APIs (in Python/Java/Scala)

Snowpark DataFrame API

- converts Python operations into SQL query, w/ lazy evaluation
- inspired from PySpark (Python for Apache Spark) DataFrame API

Snowpark Procedures and Functions

- registers Python code as stored procs and UDFs/UDTFs
- Python Worksheets as special use case
- Streamlit Apps w/ STREAMLIT object

Snowpark ML

- reimplementation of sklearn/XGBoost/LightGBM Python libs → Snowpark ML Modeling API
- Model Registry + Feature Store

Other Snowflake APIs

REST APIs

- SQL REST API = the oldest one
- Snowflake REST API = a new one
- Snowsight REST API = undocumented

Client Drivers

- Snowflake Connector for Python (Python Connector)
- JavaScript API, JDBC/ODBC, Node.js, .NET, Go, PHP

external integrations

- Snowflake Connector for Kafka/Spark

external tools

- SnowSQL and SNOW CLI + git integrations



Snowpark Sessions and Setup

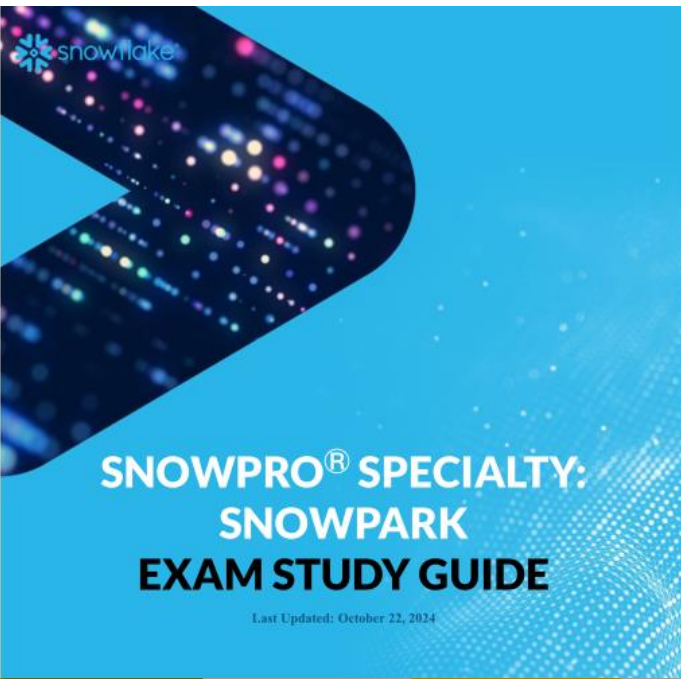
What We'll Cover (from the Study Guide)

1.2 Set-up Snowpark

- Installation: Versioning and Python environment
- Development envs: Third-party tools, Snowflake and Jupyter notebooks, VSCode

2.1 Create and manage user sessions

- Account identifiers
- Parameters for the CONNECT function
- Session creation/methods/attributes and SessionBuilder
- Authentication methods
 - Construct a dictionary
 - Key pair authentication
 - Snowflake CLI or .env parameters



Implicit Session (from Web UI Envs)

SQL Worksheets

- for SQL statements only, account/warehouse/database/schema as current context

Python Worksheets

- for Python code only, auto-converted and executed as an on-the-fly PROCEDURE

Snowflake Notebooks

- for SQL+Python, with current Snowpark session

Streamlit Apps

- for Python+Streamlit lib, with current Snowpark session, executed as a STREAMLIT object

Explicit Session (from External Envs)

- *Snowflake Extension from VSCode* → connection form + config file
- *SnowSQL and SNOW CLIs* → config file + env vars
- *Snowflake Connector for Python* → hard-coded connection params + env vars
- *Streamlit Snowflake connection* → from TOML file + *st.connection*
- *Snowpark connection*
 - **Session** class creation/methods/attributes
 - **SessionBuilder** class

Snowflake Connection Parameters

- snowflake.connector.**connect**(pars) → SnowflakeConnection obj (Python Conn)
- snowflake.snowpark.context.**get_active_session**() → Session obj (Snowpark)
- **Session**.builder.configs(pars).create() → through a **SessionBuilder**
- *connection parameters*
 - **account** = myorg-myacct or myloc (legacy) format
 - **user** = *current user name*
 - **password** = if password-based (pwd could be passed through an env var)
 - **role + warehouse** = optional (but always provided by def)
 - **database + schema** = optional (required only if specific context)
 - **authenticator** = "externalbrowser" if SSO (no pwd) / "oauth" / alt w/ **private_key**, **passcode**

Snowflake Account Identifiers

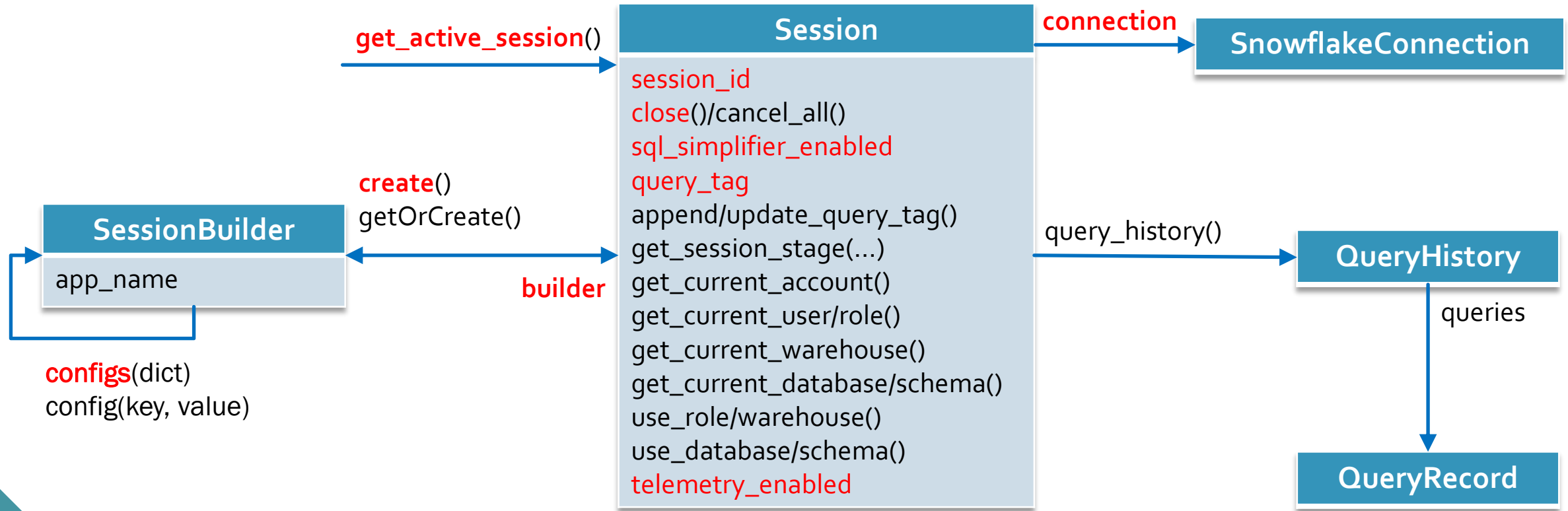
- *current account identifiers (org-based, recommended)*
 - **myorg-myacct** = preferred format, w/ **organizations** + unique **account name** within
 - `SELECT CURRENT_ORGANIZATION/ACCOUNT_NAME`
 - `ALTER ACCOUNT myacct RENAME TO myacct_new` ← rename account (within org)
- *legacy account identifiers (locator-based, discouraged)*
 - **myloc[.gov][.reg][.prov]** = w/ **locator** + optional gov/reg/provider codes
- *example account URLs*
 - <https://myorg-myacct.snowflakecomputing.com> = full URL (w/ org) to connect to Snowflake
 - <https://myloc.snowflakecomputing.com> = full URL (w/ locator) to connect to Snowflake
 - <https://app.snowflake.com/myorg/myacct/> = for the Snowflake web UI
 - <https://myloc.west-us-2.azure.snowflakecomputing.com> = old URL, w/ reg+prov
 - https://myorg-myloc_azure_west_us_2.snowflakecomputing.com = new URL, w/ prov+reg

Store the Connection Parameters

- *inline parameters* = in `snowflake.connector.connect(...)` ← for Python Connector
- *inline dictionary* = in `Session.builder.configs(...).create()` ← for Snowpark
- *environment variables* = like `SNOWSQL_PWD` for `pwd`
- *.env file* = w/ conn params, using `python-dotenv` package
- *connections.toml file* = in VSCode Snowflake ext ← in `<user_dir>/.snowflake/`
- *config.toml file* = for the new SNOW Snowflake CLI ← in `<user_dir>/.snowflake/`
- *config file* = for old SnowSQL CLI ← in `<user_dir>/.snowsql/`

Authentication Methods

- *Password-based* = unreliable, least recommended, *password* option
- *Key Pair* = for applications, w/ *key-pair rotation*
- *SSO (Single Sign-On)* = through a web browser
 - *Native SSO* = w/ Oka
 - *Federated Authentication* = w/ SAML
- *OAuth* = Snowflake or external
- *MFA (Multi-Factor Authentication)* = set `ALLOW_CLIENT_MFA_CACHING`



Session and SessionBuilder

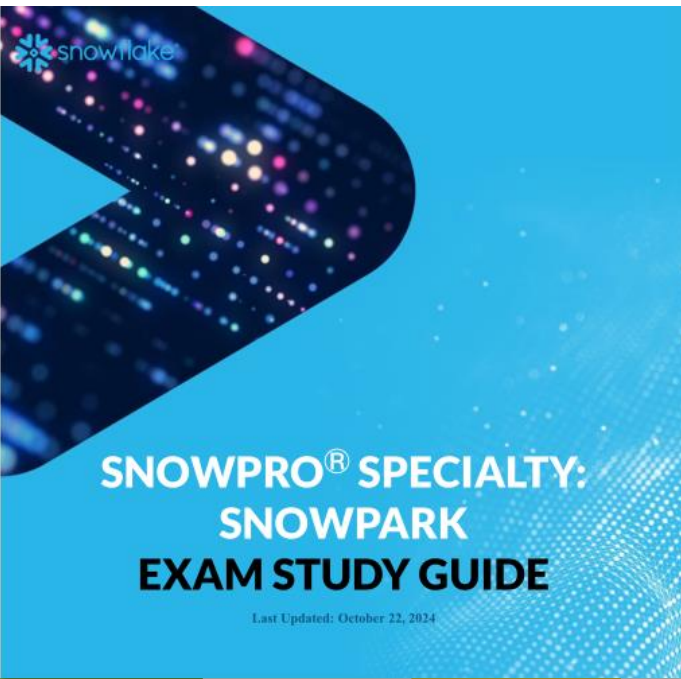


Observability in Snowpark

What We'll Cover (from the Study Guide)

4.3 Troubleshoot common errors in Snowpark

- Event tables
- Query history (SQL equivalency to help identify bottlenecks)



Logging/Metrics/Tracing

- *event tables*
 - SNOWFLAKE.TELEMTRY.EVENTS = by def
 - CREATE EVENT TABLE ... ← in a database schema, custom, to query later w/ SELECT
 - ALTER ACCOUNT SET/UNSET EVENT_TABLE [= ...] ← only one active in whole Snowflake account
- *data types*
 - log messages = detailed state messages from the code ← [logging](#) Python package, must create a named logger
 - metrics data = CPU/memory metrics generated by Snowflake
 - trace events = structured data writted from code ← [snowflake-telemetry-python](#) Python package
- *levels* → in account/session/database/schema/SP/UDF
 - LOG_LEVEL = OFF/FATAL/ERROR/WARN/INFO/DEBUG/TRACE
 - METRIC_LEVEL = NONE/ALL
 - TRACE_LEVEL = OFF/ON_EVENT/ALWAYS

Set Event Level

Set logging & tracing for:

Account

All Events

All events generated for logging, tracing, and metrics.

Status On

Traces

Record spans and trace events.

Status On

Logs

Set the severity level of log messages that you want.

FATAL

ERROR

WARN

INFO

DEBUG

TRACE

Status On

Metrics

Time series metrics for programs such as Python UDF Memory

Status On

Cancel

Save

Monitoring > Traces & Logs



QueryHistory and QueryRecord

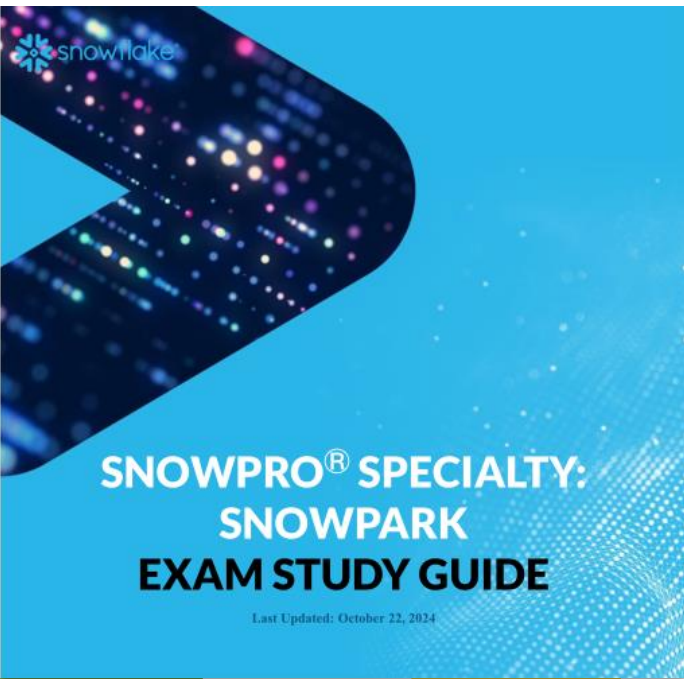


Snowpark Test Environments

What We'll Cover (from the Study Guide)

4.3 Troubleshoot common errors in Snowpark

- Snowpark Python local testing framework
- Writing tests (pytest)



Snowpark Test Environments

PyTest

- `pip install pytest`
- `src/` + `test/` folders
- compares *actual* w/ *expected* → unit tests (DataFrame) + integration tests (stored procs)
- `@pytest fixture(scope='module')` = executed before the tests

Local Testing Framework

- `pip install "snowflake-snowpark-python[localtest]"`
- `Session.builder.configs({'local_testing': True}).create()`
- `snowflake.snowpark.mock` (alt: `unittest.mock`) → fakes/simulates connection to Snowflake
- `Session.sql` → not supported, must be mocked



Snowpark for Machine Learning

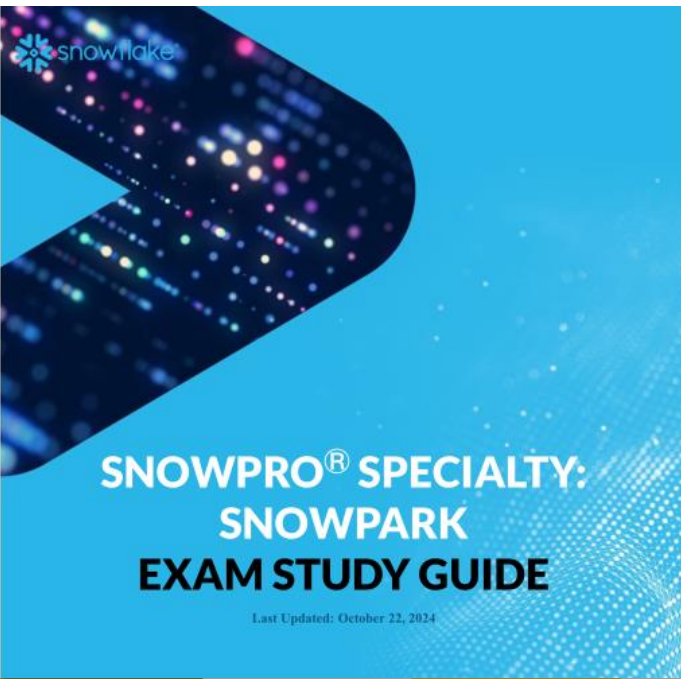
What We'll Cover (from the Study Guide)

1.1 Outline Snowpark architecture

- Types of libraries
 - Machine Learning

2.5 Operationalize Snowpark stored procedures

- Use Snowpark Python stored procedures to run workloads



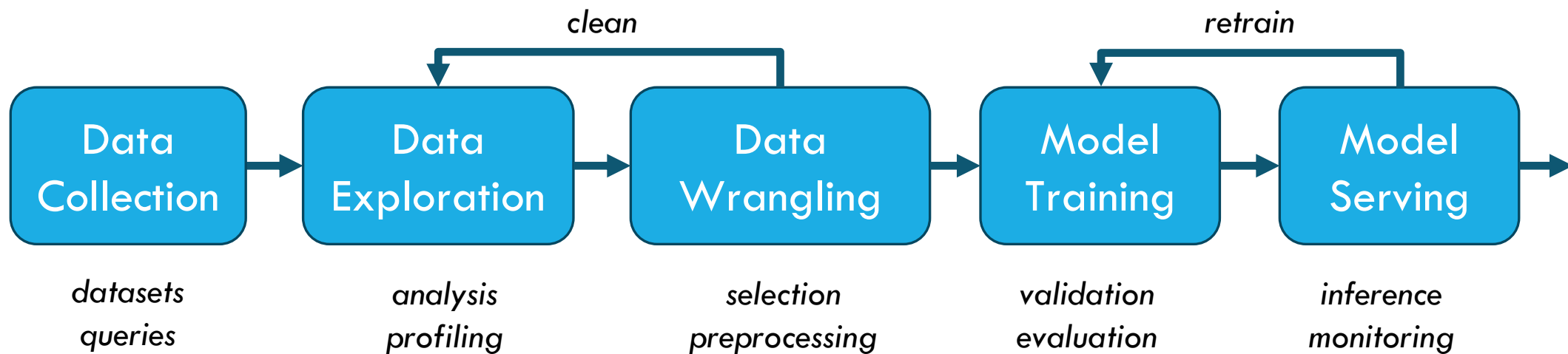
**Snowpark
DataFrame
API**

**Procedures
and Functions
API**

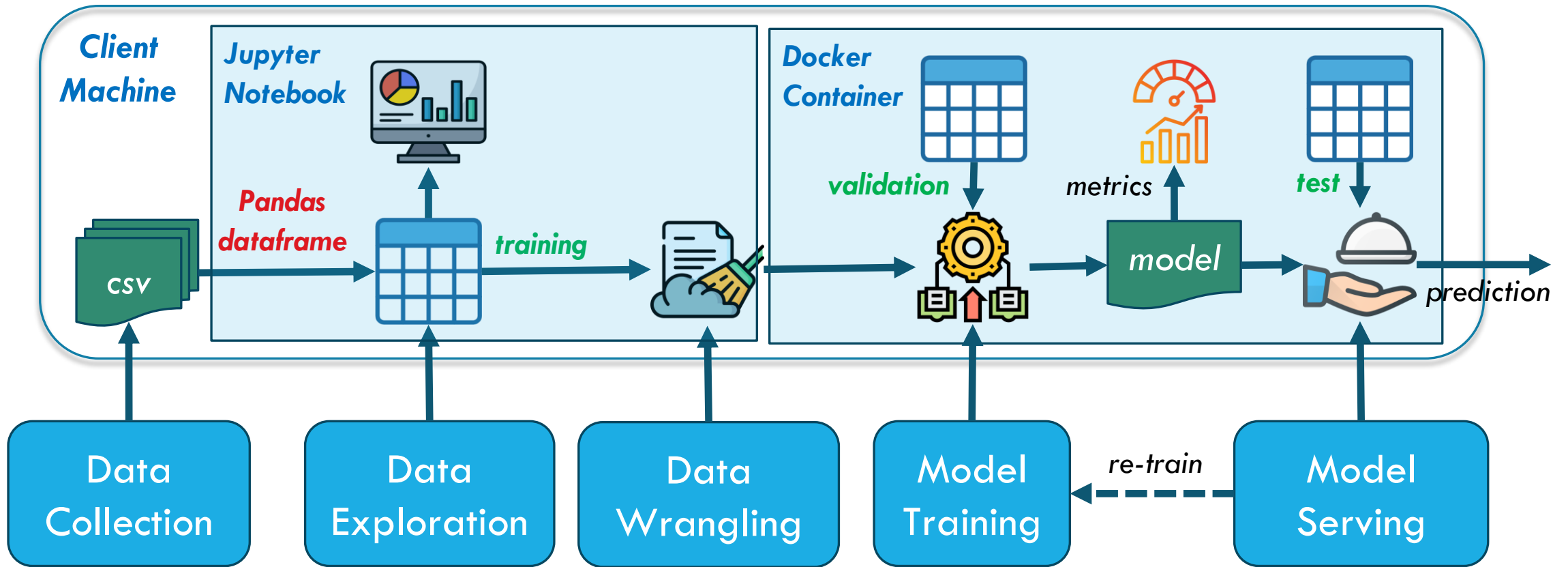
Snowpark ML
API

Snowpark
Container
Services (SPCS)

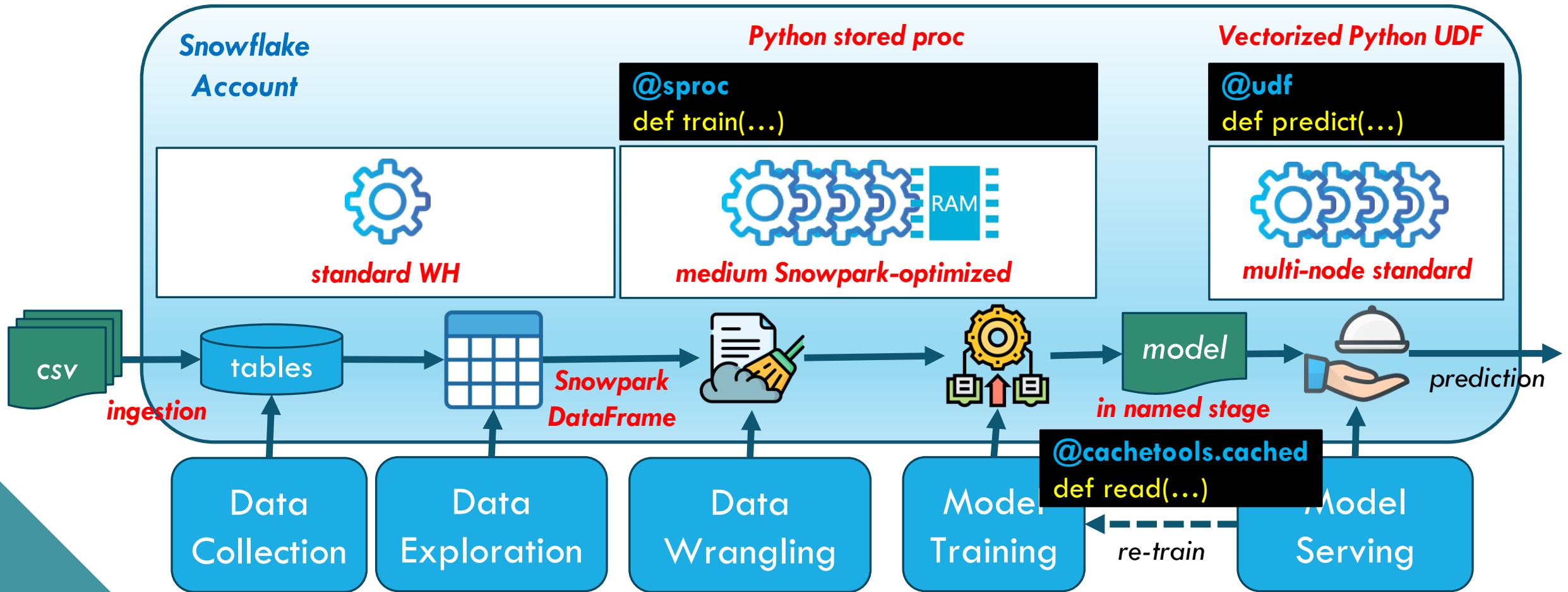
Snowpark Components



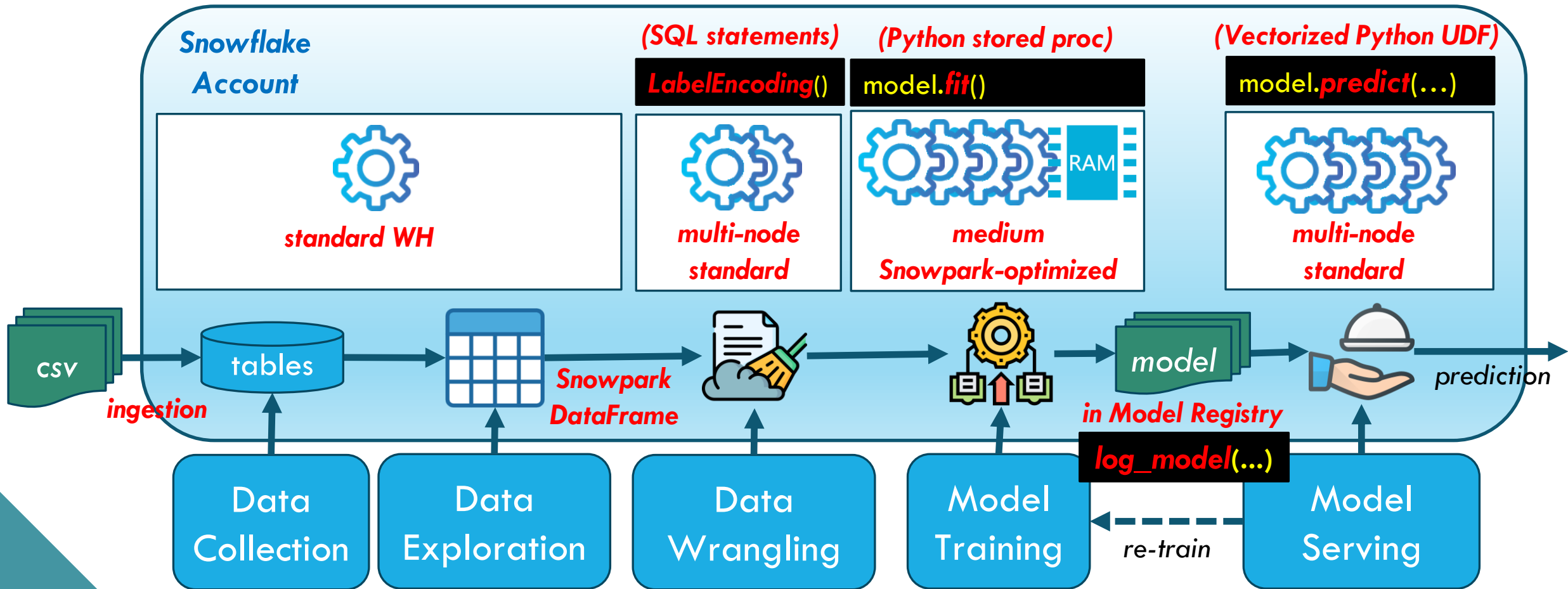
Typical ML Pipeline



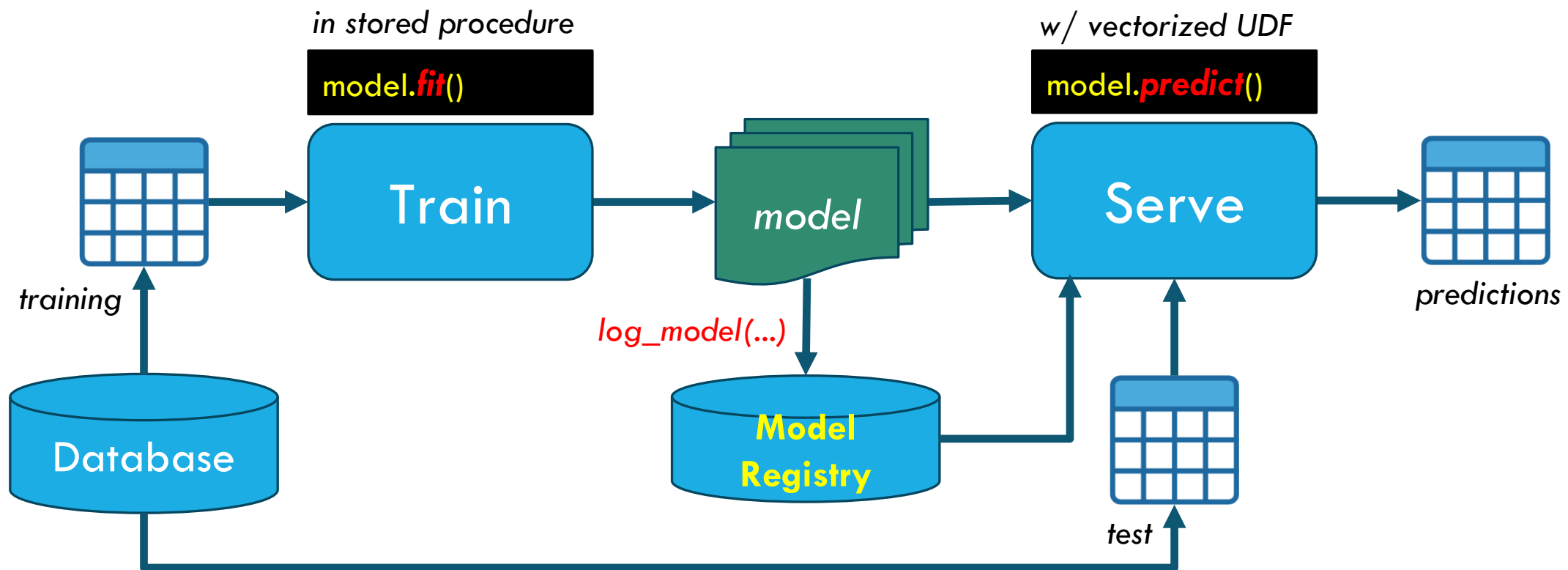
ML Pipeline before Snowpark



ML Pipeline with Snowpark



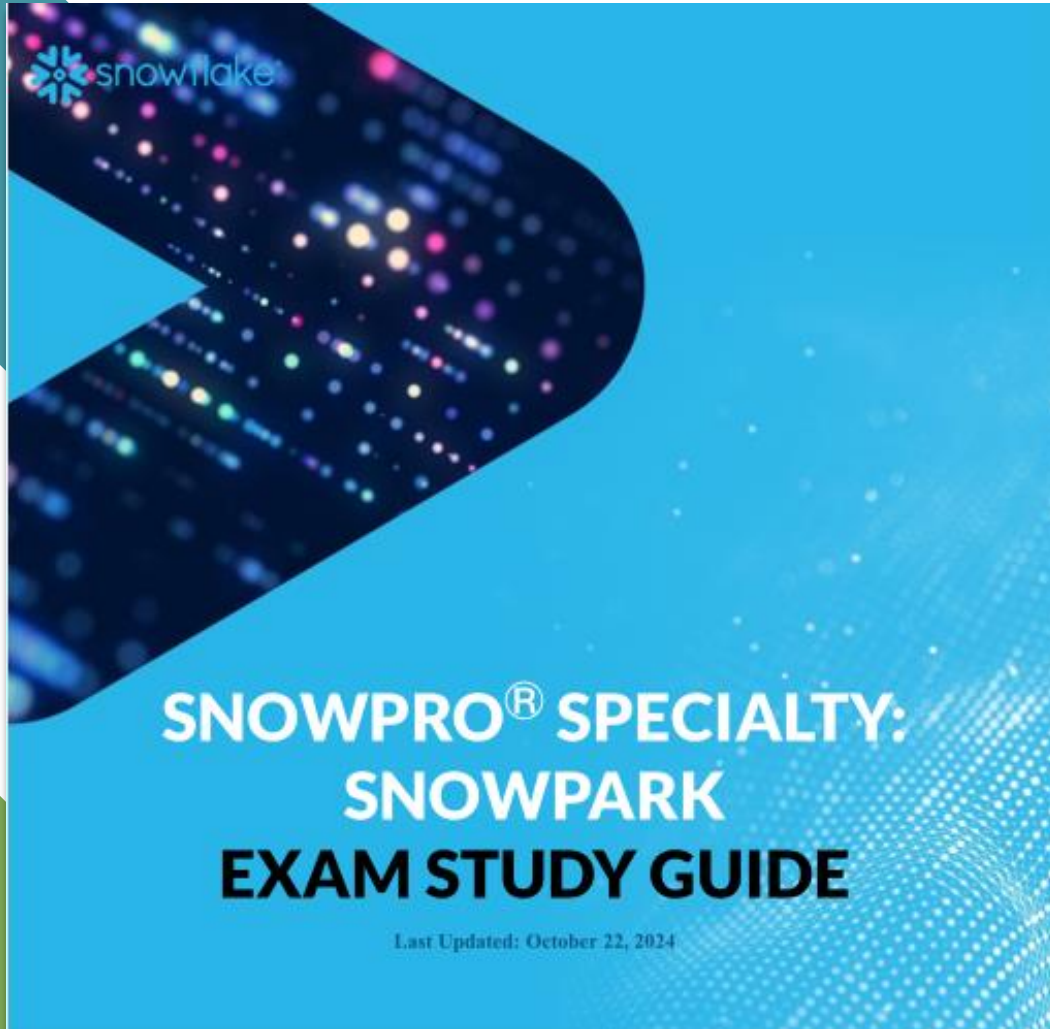
ML Pipeline with Snowpark ML



Snowpark MLOps Architecture

Snowpark ML Key Points

- separate Snowpark library ([snowpark-ml-python](#))
 - wrappers around sklearn/XGBoost/LightGBM
 - distributed fit/transform SQL processing
 - distributed HPO + performance metrics
- try to use only Snowpark DataFrames (faster than pandas)
- train models in *fit()* stored proc, w/ a medium Snowpark-optimized VW
- serve trained models from temp *predict()* vectorized UDF, w/ a multi-node std VW
- [Snowflake Model Registry](#) = to log (*log_model*) and serve trained ML models
- [Snowflake Feature Store](#) = to manage ML features



Sample Questions with Answers

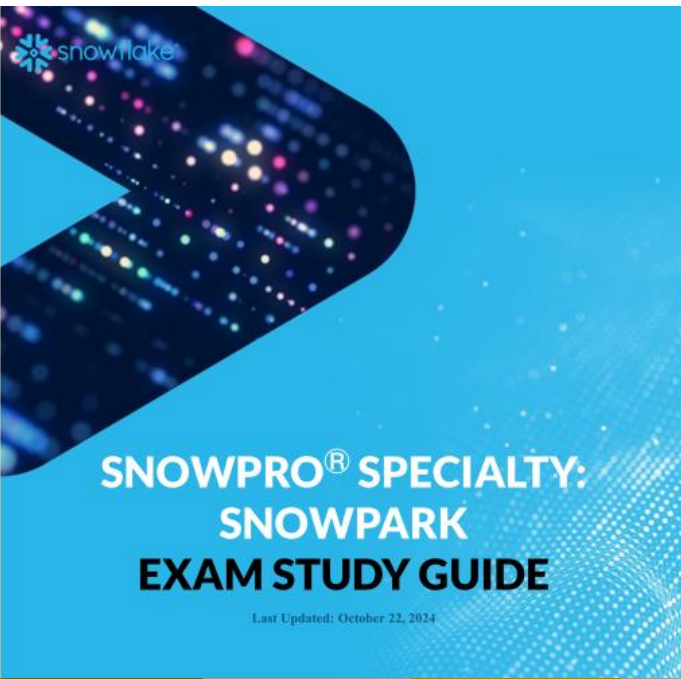
What We'll Cover (from the Study Guide)

SNOWPRO® SPECIALTY: SNOWPARK SAMPLE QUESTIONS

1. Which workload would benefit the MOST from using a Snowpark-optimized virtual warehouse?
 - A. Machine learning training
 - B. Machine learning inference
 - C. Registering a model into the Snowflake Model Registry
 - D. Creating a compute pool in Snowpark Container Services
2. A Snowpark Specialist wants to create a Python User-Defined Function (UDF) and operationalize it in Snowflake. The function will not use the `IMPORTS` clause.

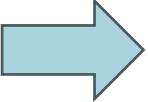
What can the Specialist do with this Python UDF in Snowflake?

- A. Share the Python UDF directly.
- B. Share a view that calls the Python UDF.
- C. Access the session object within the Python UDF.
- D. Grant the `USAGE` privilege on the Python UDF to a role.



Question 1

Which workload would benefit the MOST from using a Snowpark-optimized virtual warehouse?

- 
- A. Machine learning training
 - B. Machine learning inference
 - C. Registering a model into the Snowflake Model Registry
 - D. Creating a compute pool in Snowpark Container Services

Question 2

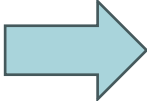
A Snowpark Specialist wants to create a Python UDF and operationalize it in Snowflake. The function will not use the *IMPORTS* clause.

What can the Specialist do with this Python UDF in Snowflake?

- A. Share the Python UDF directly.
- B. Share a view that calls the Python UDF.
- C. Access the session object within the Python UDF.
- ➡ D. Grant the *USAGE* privilege on the Python UDF to a role.

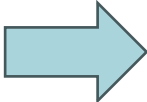
Question 3

How can a Snowpark Specialist summarize the sales quantity by product, given a DataFrame containing product sales quantities in columns named *product_id* and *quantity*?

- A. `df.sum("quantity").group_by("product_id")`
- B. `df.summarize("quantity").over("product_id")`
-  C. `df.group_by("product_id").agg(sum("quantity"))`
- D. `df.agg("quantity", type="sum").group_by("product_id")`

Question 4

A Snowpark Specialist needs to define a Python function to be used as a stored procedure. What should they consider?

- A. The function must always return a Snowpark DataFrame.
-  B. The first parameter for the function must be a Session class object.
- C. The `@sproc` decorator must always be used before the function definition.
- D. The pandas DataFrame or pandas Series object can be used as parameters for the function.

Question 5

A Snowpark Specialist developed an application that uses Snowpark for Python to interact with Snowflake tables. Users are reporting constant Multi-Factor Authentication (MFA) alerts.

What is the MOST secure method of reducing the MFA requests?

- A. Create a NETWORK POLICY for the affected users.
- ➡ B. Set the account parameter ALLOW_CLIENT_MFA_CACHING to TRUE .
- C. Allow users to add a passcode as part of their Snowpark session creation.
- D. Disable MFA temporarily for affected users using the parameter DISABLE_MFA.



SnowPro Specialty: Snowpark Certification Exam
